南方电网数字电网集团有限公司人工智能研究中心
国家人工智能应用中试基地（能源领域电力方向）
上海松应科技有限公司

2026 电力
具身智能应用大赛

详细说明文档 · 基于 ORCA 多物理场引擎平台

本文档涵盖赛前准备（硬件要求 / OrcaLab 安装 / 资产订阅）、大赛概述与系统架构、三大任务（移动巡检 / 语言指令按钮操作 / 电力工具归纳收置）详解、机器人构型、比赛规则、三层评分判定机制、ScorerClient 开发指南、任务实现方案、ORCA 仿真平台 API 参考、接入指南与常见问题排查。

文档版本：v3.2
最后更新：2026 年 8 月 5 日
适用对象：全体参赛队伍、大赛管理人员、评委

前言

南方电网具身智能大赛由南方电网与松应科技联合举办，旨在推动具身智能技术在电力行业场景中的实际应用。大赛基于 ORCA 多物理场引擎平台，为参赛队伍提供标准化、可复现的机器人操作评测环境。

本文档为大赛官方详细说明的 PDF 汇编版，完整保留原文档的章节结构与全部技术内容，共十三章：第零章介绍赛前准备工作，包括硬件要求、OrcaLab 软件安装与资产订阅；第一至五章介绍大赛背景、系统架构、三大赛题任务、机器人构型与比赛规则；第六章详解三层评分判定架构、判定原语与防作弊机制；第七、八章为开发指南与任务实现方案，包含 ScorerClient SDK、数据采集与 OpenPI 训练部署全流程；第九章为 ORCA 仿真平台 API 详细参考；第十至十二章提供一步一步接入指南、常见问题排查与附录。

参赛队伍仅需调用 4 个 API 即可非侵入式接入评分系统；所有任务、步骤与判定条件均通过 YAML 声明式配置，评分服务通过 gRPC 直连 ORCA 仿真实时判定。预祝各参赛队伍取得优异成绩。

目　录

第 0 章　大赛前准备工作 6

0.1 硬件要求 6

0.2 安装要求（Ubuntu） 7

0.3 资产订阅 8

第 1 章　大赛概述与目的 11

1.1 大赛背景 11

1.2 大赛目的 11

1.3 大赛特点 11

第 2 章　大赛系统架构 13

2.1 代码目录结构 13

2.2 整体架构图 15

2.3 两个子系统 16

2.4 技术栈 16

2.5 客户端 SDK 包结构 17

2.6 防作弊机制 17

第 3 章　大赛任务详解 19

3.1 任务一：移动巡检（20分） 19

3.2 任务二：语言指令按钮操作（40分） 20

3.3 任务三：电力工具归纳收置（40分） 22

第 4 章　大赛机器人构型 24

4.1 G1 OmniPicker（默认） 24

4.2 宇树 G1（g1_pick） 25

4.3 宇树 G1（unitree_g1 — ORCA 场景命名，仅客户端） 26

4.4 机器人配置切换 27

第 5 章　大赛规则 28

5.2 比赛流程 28

5.3 公平性保障 28

5.4 违规行为 29

第 6 章　大赛评分规则与机制 30

6.1 三层判定架构 30

6.2 评分公式 30

6.3 判定原语详解 31

6.4 诊断信息解读 35

6.5 排行榜计算 36

6.6 防作弊机制 36

第 7 章　大赛开发指南 38

7.1 环境准备 38

7.2 ScorerClient SDK 详解 38

7.3 完整示例代码 42

7.4 中央服务器 API 参考 46

第 8 章　任务实现方案详解 50

8.1 通用驱动方式 50

8.2 任务一实现方案：移动巡检 50

8.3 任务二实现方案：按钮操作 54

8.4 任务三实现方案：工具归纳 57

8.5 模型推理集成 60

第 9 章　ORCA 仿真平台 API 详细参考 62

9.1 环境定义与初始化 62

9.2 观测构建 63

9.3 动作定义与下发 65

9.4 控制器详解 66

9.5 机器人配置 68

9.7 数据采集管理器 71

9.8 第三方模型推理部署完整流程 72

9.9 奖励函数定义 74

第 10 章　一步一步接入指南 76

10.1 步骤一：安装依赖 76

10.2 步骤二：启动 ORCA 仿真环境 76

10.3 步骤三：启动评分服务 76

10.4 步骤四：编写推理代码并接入评分 79

10.5 步骤五：查看评分结果 84

10.6 步骤六：（可选）连接中央服务器 84

10.7 步骤七：提交最终成绩 86

第 11 章　常见问题排查 87

11.1 评分服务相关 87

11.2 ORCA 仿真相关 87

11.3 中央服务器相关 88

11.4 调试技巧 88

第 12 章　附录 89

12.1 ORCA 场景关键实体清单 89

12.2 配置参考 90

12.3 角色与权限矩阵 92

第 0 章

大赛前准备工作

本章介绍参赛队伍在正式比赛前需要完成的准备工作，包括硬件环境要求、OrcaLab 软件安装、以及大赛相关资产包的订阅。大赛推荐使用 **Ubuntu 系统** 版本的 OrcaLab，以获得最佳的使用体验和完整的仿真功能支持。

0.1 硬件要求

为确保 ORCA 仿真平台流畅运行，建议参赛队伍配备以下硬件环境：

硬件	推荐配置	说明
显卡	NVIDIA RTX 40/50 系列	渲染与物理仿真核心依赖，显存建议 ≥ 8GB
内存	≥ 32GB	复杂场景加载与多进程数据采集需要较大内存
存储	≥ 50GB 可用空间	资产包、训练数据、模型文件等需要充足空间
操作系统	Ubuntu 22.04 LTS / 24.04 LTS	推荐使用 Ubuntu，也可使用统信 UOS V20
网络	稳定的互联网连接	用于资产下载、依赖安装和成绩上报

驱动版本要求：

显卡系列	最低驱动版本	推荐驱动版本
NVIDIA RTX 40 系列	535.00	550+
NVIDIA RTX 50 系列	550.00	555+

系统权限要求：需要 `sudo` 权限以安装系统依赖。

0.2 安装要求（Ubuntu）

大赛推荐使用 **OrcaLab**（ORCA 仿真平台轻量版）进行开发与测试。OrcaLab 最新版本号为 **26.6.3**，以下安装命令默认下载最新版本，亦可通过指定版本号安装。

0.2.1 安装 Miniconda

```
# 下载 Miniconda 安装脚本
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

# 运行安装脚本
bash Miniconda3-latest-Linux-x86_64.sh

# 按照提示完成安装，重启终端或执行以下命令使 conda 生效
source ~/.bashrc
```

0.2.2 创建 Conda 环境

```
# 创建名为 orcalab 的 Python 3.12 环境
conda create -n orcalab python==3.12

# 激活环境
conda activate orcalab
# 激活成功时，终端提示符括号中显示 (orcalab)
```

0.2.3 安装 OrcaLab

```
# 安装 OrcaLab（推荐使用清华源或阿里源，默认源安装需挂梯子且网速较慢）
pip install orca-lab -i https://pypi.tuna.tsinghua.edu.cn/simple

# 如需指定版本号安装（例如安装 26.6.3 版本）：
# pip install orca-lab==26.6.3 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

0.2.4 启动 OrcaLab

```
# 首次启动（会自动安装 Python 依赖包）
orcalab
```

首次运行 OrcaLab 会自动安装 Python 依赖包，请耐心等待。启动后会出现场景与布局选择界面。

0.2.5 验证安装

```
# 检查 OrcaLab 版本
pip show orca-lab

# 启动 OrcaLab 验证
orcalab
```

0.2.6 OrcaLab 升级与卸载

```
# 升级到最新版本（需要添加 --upgrade 参数）
pip install --upgrade orca-lab

# 卸载 OrcaLab
conda deactivate                    # 退出 conda 环境
conda env remove -n orcalab         # 删除 conda 环境
```

0.3 资产订阅

ORCA 资产库（https://simassets.orca3d.cn/）是大赛场景与机器人模型的分发平台。参赛队伍需要在资产库中订阅大赛相关的资产包，订阅后 OrcaLab 会在下次启动时自动下载并同步资产。

0.3.1 用户注册与登录

1. 打开资产库页面：**https://simassets.orca3d.cn/**
2. 已注册用户直接登录，未注册用户需先完成注册
3. 注册方式：
   - **手机/邮箱注册**：输入手机号 → 验证码 → 完善账号信息（姓名、用户名、密码、组织名称）
   - **第三方账号注册**：支持微信和 GitHub 第三方账号注册
4. 注册成功后会自动登录，并弹出资产库界面

0.3.2 订阅大赛资产包

登录资产库后，在资产中心搜索并订阅以下资产包：

1. 大赛场景资产包

资产包名称	说明	包含内容

`SouthGrid_Competetion_2026`	南方电网 2026 具身智能大赛场景	电气柜、按钮面板、工具箱、检查点 A/B/C、电力工具等

2. 机器人模型资产包

资产包名称	对应机器人	说明
`G1_Picker_SouthGrid`	**宇树 G1 机器人**	宇树科技 G1 双足人形机器人，浮动基座 + 12 个腿部关节，双臂末端操作
`G1_omnipicker`	**智元精灵 G1 机器人**	智元精灵 G1 轮式人形机器人，轮式底盘（阿克曼转向）+ 双臂末端操作，大赛默认机器人

> **注意**：两个机器人资产包均需订阅，以确保参赛队伍可根据任务需求选择使用不同的机器人构型。大赛默认使用 `G1_omnipicker`（智元精灵 G1 轮式机器人）。

0.3.3 订阅操作步骤

1. 在资产库中搜索资产包名称（如 `SouthGrid_Competetion_2026`）
2. 进入资产包详情页面，点击 **【订阅】** 按钮
3. 订阅成功后，资产包会出现在 **个人中心** → **已订阅资产** 列表中
4. 同样方式订阅 `G1_Picker_SouthGrid` 和 `G1_omnipicker` 两个机器人资产包

0.3.4 同步资产到本地

订阅完成后，需要重启 OrcaLab 以触发资产同步下载：

```
# 如果 OrcaLab 正在运行，先关闭
# 然后重新启动
conda activate orcalab
orcalab
```

启动后，会显示 **【正在同步资产包...】** 提示，OrcaLab 会自动下载新订阅的资产包并更新已订阅的资产包。

> **注意**：带 **【有场景】** 标签的资产包（如 `SouthGrid_Competetion_2026`）同步完成后，可在场景选择界面中看到对应场景，选择场景后点击 **【打开】** 按钮即可进入大赛仿真环境。

0.3.5 网络问题处理

如果遇到 pip 安装下载速度慢、资产同步失败等问题：

- **pip 安装慢**：使用清华源 `-i https://pypi.tuna.tsinghua.edu.cn/simple` 或阿里源
- **资产下载失败**：检查网络连接，确保能访问 `https://simassets.orca3d.cn/`；如已下载资产，可使用离线启动模式
- **资产库访问速度慢**：建议使用稳定的网络环境，避免使用代理

第 1 章

大赛概述与目的

1.1 大赛背景

南方电网具身智能大赛（以下简称"大赛"）由南方电网与松应科技联合举办，旨在推动具身智能技术在电力行业场景中的实际应用。大赛基于 **ORCA 多物理场引擎平台**（后端支持 MuJoCo 等物理引擎），为参赛队伍提供一个标准化、可复现的机器人操作评测环境。

1.2 大赛目的

1. **推动具身智能技术落地**：将前沿的机器人学习算法（如模仿学习、强化学习、大模型驱动的机器人控制）应用于电力行业真实场景。
2. **建立标准化评测基准**：通过 ORCA 仿真平台提供统一的、可复现的评测环境，确保比赛公平公正。
3. **促进产学研合作**：为高校、研究机构和企业提供技术交流平台，加速具身智能技术在电力行业的产业化进程。
4. **培养复合型人才**：培养同时具备机器人控制、人工智能算法、电力行业知识的复合型技术人才。

1.3 大赛特点

特点	说明
非侵入式评分	参赛队伍仅需调用 4 个 API 即可接入评分系统，无需修改推理代码核心逻辑
声明式任务配置	所有任务、步骤、判定条件通过 YAML 文件定义，无需编写 Python 判定代码
实时 ORCA 状态判定	评分服务通过 gRPC 直连 ORCA 仿真，实时查询机器人状态并自动判定
多角色权限管理	支持管理员、评委、开发者三种角色，完整的鉴权和权限控制
完整 Web UI	提供排行榜、步骤成功率热力图、在线文档编辑、视频回放等可视化功能

双重防作弊机制	评分服务以二进制可执行文件分发 + SHA-256 配置文件哈希校验，防止评分规则被篡改

第 2 章

大赛系统架构

2.1 代码目录结构

大赛评分系统拆分为两个独立的代码目录，分别对应本地开发端和中央服务器端：

```
OrcaManipulation/src/examples/
├── competition_scoring_client/       # 本地开发端（参赛队伍使用）
│   ├── scorer_service/               #   评分服务核心（端口 9000）
│   │   ├── main.py                   #     服务入口，含 SHA-256 配置哈希校验
│   │   ├── server.py                 #     FastAPI + 内嵌 Web UI
│   │   ├── engine.py                 #     评分引擎（任务生命周期、帧采样、判定）
│   │   ├── orca_observer.py          #     ORCA 状态观察器（gRPC 连接）
│   │   ├── frame_oracle.py           #     帧数据存储与回溯
│   │   ├── video_recorder.py         #     视频录制（屏幕捕获 + ORCA 内置）
│   │   └── tests/                    #     评分服务测试（诊断脚本、集成测试）
│   ├── scorer_client/                #   旧导入名兼容包（→ orca_scorer_client）
│   ├── orca_scorer_client/           #   轻量客户端 SDK 包（零外部依赖）
│   ├── sdk/orca_competition/         #   核心 SDK（原语、条件、解析器、动态探测）
│   │   ├── primitives.py             #     环境原语层（Layer 1）
│   │   ├── conditions.py             #     谓词组合层（Layer 2）
│   │   ├── condition_parser.py       #     YAML 解析层（Layer 3）
│   │   ├── robot_detector.py         #     ORCA 场景机器人动态探测
│   │   ├── client.py                 #     CompetitionClient（旧架构兼容）
│   │   ├── probes.py                 #     非侵入探针基类
│   │   ├── registry.py               #     探针注册表
│   │   ├── transport.py              #     HTTP 传输层
│   │   ├── local_store.py            #     本地存储
│   │   ├── models.py                 #     数据模型（Pydantic）
│   │   └── cli.py                    #     命令行接口
│   ├── tasks/                        #   任务链构建
│   ├── probes/                       #   过程性探针库（按钮、抓取、导航、电压等）
│   ├── configs/                      #   任务/机器人/队伍/评委配置
│   │   ├── tasks.yaml                #     三任务声明式配置
│   │   ├── robots.yaml               #     机器人配置（含 unitree_g1 适配）
│   │   ├── teams.yaml                #     预置队伍注册表
│   │   ├── judges.yaml               #     评委配置
│   │   └── config.hash               #     配置文件 SHA-256 哈希（防篡改）
│   ├── dashboard/                    #   本地评分面板 Web UI
│   ├── examples/                     #   使用示例
│   ├── tests/                        #   集成测试与任务测试脚本
│   │   ├── task1_inspection/         #     任务一测试（含 g1_pick/g1_omnipicker）
│   │   ├── task2_button_press/       #     任务二测试
│   │   └── task3_tool_storage/       #     任务三测试
│   ├── nav_task1_test/               #   任务一导航测试（独立导航控制器）
│   ├── build_scorer_service.py       #   PyInstaller 打包脚本（生成二进制分发）
```

```
│   └── scorer_service.spec           #   PyInstaller spec 文件
│
├── competition_scoring_server/       # 中央服务器端（管理员部署）
│   ├── app/                          #   FastAPI 应用
│   │   ├── main.py                   #     应用入口（含品牌管理、文档 API）
│   │   ├── auth.py                   #     JWT 鉴权（HS256，零外部依赖）
│   │   ├── cli.py                    #     命令行管理工具
│   │   ├── api/                      #     REST API 路由
│   │   │   ├── routes.py             #       主路由聚合
│   │   │   ├── judges.py             #       评委相关 API
│   │   │   ├── uploads.py            #       文件上传 API
│   │   │   └── oss_uploads.py        #       阿里云 OSS 上传
│   │   ├── models/                   #     Pydantic/SQLAlchemy 数据模型
│   │   │   └── schemas.py            #       队伍、任务、上传、评分等模型
│   │   ├── scoring/                  #     评分聚合引擎
│   │   │   └── engine.py             #       排行榜计算、分数聚合
│   │   ├── storage/                  #     存储层
│   │   │   ├── sqlite_storage.py     #       SQLite 存储（WAL 模式）
│   │   │   └── postgresql_storage.py #       PostgreSQL 存储（可选）
│   │   └── cache/                    #     缓存层
│   │       └── redis_cache.py        #       Redis 缓存
│   ├── configs/                      #   默认配置
│   │   ├── tasks.yaml                #     三任务配置（与 client 同步）
│   │   ├── robots.yaml               #     机器人配置（g1_omnipicker + g1_pick）
│   │   ├── teams.yaml                #     预置队伍
│   │   ├── judges.yaml               #     评委配置
│   │   └── branding.json             #     品牌化配置（Logo、标题）
│   ├── web/                          #   前端页面
│   │   ├── index.html                #     主面板
│   │   ├── login.html                #     登录页
│   │   ├── guide.html                #     大赛指南
│   │   ├── dev_guide.html            #     开发者指南
│   │   └── admin_docs.html           #     管理员文档
│   ├── tests/                        #   服务端测试（E2E、上传、导入导出等）
│   ├── deploy/                       #   部署脚本
│   │   └── aliyun/                   #     阿里云部署（Terraform + 多种脚本）
│   ├── TEAM_GUIDE.md                 #   大赛细则
│   ├── DEV_GUIDE.md                  #   开发者接入指南
│   ├── ADMIN_GUIDE.md                #   管理员运维文档
│   ├── .env.example                  #   环境变量示例
```

```
│   └── deploy.sh / quick_deploy.sh   #   一键部署脚本
│
└── dataCollection/                   # 数据采集脚本（独立目录）
```

2.2 整体架构图


南方电网具身智能大赛评分平台
competition_scoring_client/ 本地开发端
ScorerService
(端口 9000)
- 步骤判定引擎
- 20Hz 后台采样
- 帧回溯自动判定
- 视频录制（屏幕捕获）
- SHA-256 配置校验
gRPC
ORCA 仿真引擎
(端口 50051)
- G1 机器人模型
- 场景 site/body
- 物理仿真
HTTP (localhost:9000)
参赛队伍推理代码
(ScorerClient SDK)
本地评分面板 (dashboard/)
http://127.0.0.1:9000/ — 实时步骤状态、得分、诊断信息
防作弊层：PyInstaller 二进制分发 + config.hash 哈希校验
HTTP 上报结果
competition_scoring_server/ 中央服务器端
中央管理平台 (FastAPI, 端口 8000)
JWT 鉴权
三角色
REST API
CRUD
评分引擎
排行榜
WebSocket
实时推送
SQLite/PostgreSQL 数据库
Web UI 前端 (web/)
/login.html 登录页（管理员/评委/开发者）
/index.html 主面板（角色仪表盘）
/guide.html 大赛指南（公开，管理员可编辑）
/dev_guide.html 开发者指南（公开）
/admin_docs.html 管理员文档（仅管理员可访问）

2.3 两个子系统

子系统	所在目录	端口	职责	必需性
ScorerService（评分判定服务）	`competition_scoring_client/scorer_service/`	9000	步骤判定引擎，通过 gRPC 直连 ORCA 仿真，20Hz 采样 + 帧回溯自动判定	**必需**（评分判定核心）
中央管理平台	`competition_scoring_server/`	8000	排行榜、队伍/评委管理、数据存储、Web UI 展示、品牌管理	必需（管理与展示）

2.4 技术栈

组件	技术
Web 框架	FastAPI (Python)
数据库	SQLite 3（WAL 模式） / PostgreSQL（可选）
鉴权	自实现 JWT（HS256，零外部依赖）
实时通信	WebSocket
仿真通信	gRPC（同步直连）
前端	原生 HTML/JS + marked.js (Markdown 渲染)
配置格式	YAML
物理引擎	ORCA 多物理场引擎（支持 MuJoCo 等后端）
二进制分发	PyInstaller（打包 scorer_service 为独立可执行文件）
缓存	Redis（可选）

对象存储	阿里云 OSS（可选）

2.5 客户端 SDK 包结构

参赛队伍主要通过以下两个 Python 包与评分服务交互：

包名	路径	说明
`orca_scorer_client`	`competition_scoring_client/orca_scorer_client/`	**推荐使用**：轻量客户端 SDK，零外部依赖，纯标准库实现
`scorer_client`	`competition_scoring_client/scorer_client/`	旧导入名兼容包，实际导入 `orca_scorer_client`，保持向后兼容

```
# 推荐的新导入方式
from orca_scorer_client import ScorerClient

# 兼容的旧导入方式（功能相同）
from scorer_client import ScorerClient
```

2.6 防作弊机制

评分系统采用双重防作弊机制，确保比赛公平公正：

2.6.1 二进制分发

评分服务（`scorer_service`）通过 **PyInstaller** 打包为独立可执行文件，而非分发源代码：

- 打包命令：`python build_scorer_service.py`
- 输出：`dist/scorer_service/scorer_service.exe`（Windows）或 `scorer_service`（Linux/macOS）
- 打包内容：`scorer_service/` + `sdk/orca_competition/` + `tasks/` 的所有 Python 模块
- 数据文件：`configs/tasks.yaml`、`configs/robots.yaml`、`configs/config.hash`
- ScorerClient 优先启动二进制文件，二进制不可用时回退到源码模式

2.6.2 SHA-256 配置文件哈希校验

关键评分规则文件（`tasks.yaml`、`robots.yaml`）在打包时生成 SHA-256 哈希值，存储于 `configs/config.hash`。评分服务启动时自动校验：

- 校验通过：正常启动
- 校验失败：打印警告，但服务仍可运行（默认模式）
- 严格模式 `--strict-config`：校验失败直接退出，服务拒绝启动

这确保了参赛队伍无法通过修改评分规则文件来获得不公平优势。

第 3 章

大赛任务详解

大赛共设**三个任务**，总分 **100 分**。所有任务的失败策略均为 `continue`（某步骤失败不影响后续步骤，各步骤独立计分）。

所有任务配置通过 `configs/tasks.yaml` 文件声明式定义，使用 `${robot.xxx}` 占位符引用机器人参数，系统自动从 `configs/robots.yaml` 中加载对应配置。

3.1 任务一：移动巡检（20分）

任务 ID：`task1_inspection`
满分：20 分（权重 0.20）
步骤数：5 步

任务目标

控制 G1 机器人从起点出发，依次经过检查点 A、B，最终到达目标电力设备柜 C，全程保持平稳移动且无碰撞。

关键实体

实体	ORCA 名称	用途
机器人基座	`${robot.base_body}` → 动态替换	用于位置和速度判定
检查点 A	`Static_Location_A_site`	路径经过点 A
检查点 B	`Static_Location_B_site`	路径经过点 B
目标设备柜	`Static_Location_C_site`	最终到达点

步骤详情

步骤	step_id	分值	判定条件	类型
1. 启动移动	`start_move`	3 分	基座线速度 >= 0.05 m/s，持续 1 秒	速度判定

2. 经过检查点 A	`pass_checkpoint_a`	4 分	基座进入 Location A 区域（半径 0.6m）	区域判定
3. 经过检查点 B	`pass_checkpoint_b`	4 分	基座进入 Location B 区域（半径 0.6m）	区域判定
4. 到达目标设备柜	`arrive_panel`	4 分	基座进入 Location C 区域（半径 0.6m）	区域判定
5. 平稳移动	`smooth_motion`	5 分	全程无碰撞 + 速度曲线平滑（jerk <= 2.0）	全程性判定

第 5 步特殊说明

`smooth_motion` 标记了 `whole_step: true`，表示这是一个**全程性判定**。评分服务会在整个步骤执行期间持续采样（20Hz），而非仅在 `end_step()` 时判定一次。这意味着参赛队伍需要在整个移动过程中保持平稳，不能仅在最后时刻满足条件。

距离计算方式

所有 `point_in_region` 判定使用 **XY 平面欧氏距离**（忽略 Z 轴高度差），公式为：

```
sqrt((x1 - x2)^2 + (y1 - y2)^2) <= radius
```

3.2 任务二：语言指令按钮操作（40分）

任务 ID：`task2_button_press`
满分：40 分（权重 0.40）
步骤数：4 步（4 个独立子任务）

任务目标

正确解析自然语言指令，识别目标颜色按钮的位置，控制末端执行器移动到对应按钮位置完成模拟按压。

关键实体

实体	ORCA 名称

末端执行器	`${robot.ee_site}` → 动态替换（如 `g1_omnipicker_ee_center_site_l`）

按钮映射

颜色	step_id	分值	按钮 site 名称	判定半径
红色	`press_red_button`	10 分	`Group_Static_ElectricalCabinet_button01_site`	0.10m
绿色	`press_green_button`	10 分	`Group_Static_ElectricalCabinet_button02_site`	0.10m
蓝色	`press_blue_button`	10 分	`Group_Static_ElectricalCabinet_button03_site`	0.10m
黄色	`press_yellow_button`	10 分	`Group_Static_ElectricalCabinet_button04_site`	0.10m

判定条件

每个按钮独立判定：末端执行器 site 与对应按钮 site 的 **XY 平面欧氏距离 <= 0.10m** 即为成功。

子任务独立性

四个按钮子任务相互独立，失败策略为 `continue`。即使某个按钮操作失败，其他按钮仍可得分。

动态按钮区域判定（`target_button_region`）

SDK 提供 `TargetButtonRegion` 原语，支持根据队伍通过 `set_extra("target_color", "red")` 设置的上下文动态选择对应按钮。按钮颜色映射：

```
red    → button01_site
green  → button02_site
blue   → button03_site
yellow → button04_site
```

> **注意**：当前 ORCA 场景中可能缺少 `button03_site`（蓝色按钮），该步骤预期会失败。请以实际 ORCA 场景中的 site 列表为准。

3.3 任务三：电力工具归纳收置（40分）

任务 ID：`task3_tool_storage`
满分：40 分（权重 0.40）
步骤数：6 步

任务目标

控制机器人到达工具箱作业位姿，依次将 5 件电力工具拾取并归位到工具箱中。

关键实体

实体	ORCA 名称	用途
工具箱基座	`Group_Interactive_ToolBox_base_site`	工具归位目标区域
末端执行器	`${robot.ee_site}` → 动态替换	夹爪判定参考点
夹爪手指 A	`${robot.finger_a}` → 动态替换	抓取判定
夹爪手指 B	`${robot.finger_b}` → 动态替换	抓取判定

步骤详情

步骤	step_id	分值	工具 site 名称	判定条件
1. 到达作业位姿	`reach_tool_table`	5 分	-	基座进入工具箱区域（半径 0.6m）
2. 验电笔归位	`pick_voltage_pen`	7 分	`Group_Interactive_TestPencil_task_testpencil_testpoint_site`	三条件 AND
3. 螺丝刀归位	`pick_screwdriver`	7 分	`Group_Interactive_Screwdriver_task_screwdriver_site`	三条件 AND
4. 扳手归位	`pick_wrench`	7 分	`Group_Interactive_Spanner_task_spanner_site`	三条件 AND

5. 小手电筒归位	`pick_flashlight`	7 分	`Group_Interactive_Flashlight_task_flashlight_site`	三条件 AND
6. 电工刀归位	`pick_electrician_knife`	7 分	`Group_Interactive_ElectriciansKnife01_task_electriciansknife01_site`	三条件 AND

工具归位三条件（必须同时满足）

1. **位置条件**：工具 site 在工具箱基座区域（`Group_Interactive_ToolBox_base_site`）半径 **0.3m** 内
2. **释放条件**：工具**未被**夹爪抓取（`NOT is_grasped`）
3. **稳定条件**：工具在工具箱区域内**持续 1 秒以上**

`is_grasped` 判定逻辑

检查工具 site 与末端执行器 site 的距离是否 <= 0.10m（proximity），且两个夹爪关节的宽度 <= 0.05m（closed_threshold，表示夹爪闭合）。如果工具靠近末端且夹爪闭合，则判定为"被抓取中"。

第 4 章

大赛机器人构型

大赛支持**三种**机器人配置，参赛队伍可根据任务需求选择使用。默认使用 **G1 OmniPicker**。系统通过 `robots.yaml` 中的 `robot_profiles` 统一管理，并支持 **ORCA 场景动态探测**（`RobotDetector`）自动识别当前场景中的机器人类型。

> **注意**：`unitree_g1` 配置仅在客户端 `competition_scoring_client/configs/robots.yaml` 中存在，用于适配 ORCA 场景中以 `unitree_humanoid_robot_1` 为前缀命名的 G1 机器人。服务器端 `competition_scoring_server/configs/robots.yaml` 仅包含 `g1_omnipicker` 和 `g1_pick` 两种配置。

4.1 G1 OmniPicker（默认）

参数	值	说明
基础 body	`g1_omnipicker_body_link1`	机器人基座，用于位置和速度判定
自由关节	`g1_omnipicker_free_joint`	底盘自由移动关节
末端执行器 site	`g1_omnipicker_ee_center_site_l`	左臂末端中心点
夹爪手指 A	`g1_omnipicker_idx31_gripper_l_inner_joint1`	左手指关节 1
夹爪手指 B	`g1_omnipicker_idx39_gripper_l_inner_joint2`	左手指关节 2
移动方式	轮式底盘（阿克曼转向）	4 个执行器（2 轮驱动 + 2 转向）
驱动轮名称	`wheel_fl_joint_mctrl`, `wheel_fr_joint_mctrl`	前驱

转向关节	`wheel_fl_steer_joint_pctrl`, `wheel_fr_steer_joint_pctrl`	前轮转向
最大速度	30 m/s	轮速
最大转向角	0.6 rad	阿克曼转向
轴距	0.42 m	前后轮距
轮距	0.26 m	左右轮距
body 子串	`omnipicker_body`	用于动态匹配
ee site 子串	`omnipicker_ee`	用于动态匹配

4.2 宇树 G1（g1_pick）

参数	值	说明
基础 body	`g1_pick_pelvis`	骨盆作为根 body
自由关节	`g1_pick_floating_base_joint`	浮动基座关节
末端执行器 site	`g1_pick_left_palm`	左手掌
右末端执行器	`g1_pick_right_palm`	右手掌
夹爪手指 A	`g1_pick_left_hand_thumb_0_joint`	左手拇指关节
夹爪手指 B	`g1_pick_left_hand_index_0_joint`	左手食指关节
移动方式	双足行走（bipedal）	12 个腿部关节
腿部关节	左/右髋 pitch/roll/yaw + 膝 + 踝 pitch/roll	步态控制
步长	0.3 m	单步长度
抬腿高度	0.05 m	步态高度
行走速度	0.5 m/s	设计行走速度
平衡阈值	0.05	平衡判定
body 子串	`g1_pick_pelvis`	用于动态匹配
ee site 子串	`g1_pick_left_palm`	用于动态匹配

4.3 宇树 G1（unitree_g1 — ORCA 场景命名，仅客户端）

部分 ORCA 场景中机器人以 `unitree_humanoid_robot_1` 为前缀命名，此配置适配该场景。

参数	值	说明
基础 body	`unitree_humanoid_robot_1_pelvis`	骨盆作为根 body
自由关节	`unitree_humanoid_robot_1_floating_base_joint`	浮动基座关节
末端执行器 site	`unitree_humanoid_robot_1_left_palm`	左手掌
右末端执行器	`unitree_humanoid_robot_1_right_palm`	右手掌
夹爪手指 A	`unitree_humanoid_robot_1_left_hand_thumb_0_joint`	左手拇指关节
夹爪手指 B	`unitree_humanoid_robot_1_left_hand_index_0_joint`	左手食指关节
移动方式	双足行走（bipedal）	12 个腿部关节
步长	0.3 m	单步长度
抬腿高度	0.05 m	步态高度
行走速度	0.5 m/s	设计行走速度
body 子串	`unitree_humanoid_robot_1_pelvis`	用于动态匹配
ee site 子串	`unitree_humanoid_robot_1_left_palm`	用于动态匹配

4.4 机器人配置切换

任务配置文件中使用 `${robot.xxx}` 占位符引用机器人参数，系统会自动从 `robots.yaml` 中加载对应配置。默认使用 `g1_omnipicker`，可通过以下方式切换：

1. **修改 `robots.yaml` 中的 `default_robot` 字段**：全局切换默认机器人
2. **指定 `--robot-id` 参数**：启动评分服务时覆盖默认机器人
3. **ORCA 场景动态探测**：`RobotDetector` 自动识别场景中的机器人类型，优先级最高

动态探测机制

`RobotDetector`（`sdk/orca_competition/robot_detector.py`）通过分析 ORCA 场景中的 body/site/joint 命名规律自动识别机器人类型：

- 特征匹配：`omnipicker_body` → G1 OmniPicker，`g1_pick_pelvis` → 宇树 G1
- 至少匹配 2 个特征才确认类型
- 探测成功的字段优先使用，失败的字段回退到配置文件默认值

第 5 章

大赛规则

5.1 基本规则

1. **仿真环境**：所有任务在 ORCA 多物理场引擎平台中完成。
2. **编程语言**：参赛队伍可使用 Python 编写推理和控制代码，系统提供 Python SDK（ScorerClient）。
3. **评分方式**：评分由评分服务自动完成，无需人工干预。评分服务通过 gRPC 直连 ORCA 仿真，实时查询机器人状态并进行判定。
4. **多次尝试**：参赛队伍可对同一任务进行多次尝试，历史记录会保留。系统会取各任务的最佳得分率计算最终成绩。
5. **失败策略**：所有任务的失败策略均为 `continue`，即某步骤失败不影响后续步骤，各步骤独立计分。
6. **代码独立性**：参赛队伍的推理代码与评分代码完全解耦，仅通过 4 个 API 调用与评分服务交互。
7. **防作弊规则**：评分服务以二进制可执行文件形式分发，评分规则文件受 SHA-256 哈希校验保护，参赛队伍不得修改评分逻辑。

5.2 比赛流程

1. **队伍注册**：管理员通过中央服务器为每支队伍分配 `team_id` 和 `team_token`。
2. **本地开发**：参赛队伍在本地启动 ORCA 仿真 + 评分服务，编写推理代码进行开发调试。
3. **本地测试**：参赛队伍通过 ScorerClient SDK 在本地完成评分测试，确认各步骤得分。
4. **结果上报**：参赛队伍通过中央服务器 API 将尝试结果上报到中央服务器。
5. **最终评审**：评委通过中央服务器查看各队伍成绩，可进行人工打分（0-100 分）。

5.3 公平性保障

- **统一仿真环境**：所有参赛队伍使用相同的 ORCA 仿真场景和机器人模型，确保环境一致。
- **自动评分**：步骤判定由评分服务自动完成，基于物理仿真状态实时判定，排除人为因素。
- **声明式判定条件**：所有判定条件在 YAML 配置文件中声明，公开透明。
- **诊断信息透明**：每次判定都会返回详细的诊断信息，包括实际测量值和期望阈值，队伍可自行验证。
- **二进制分发 + 哈希校验**：评分服务以二进制分发，评分规则文件受 SHA-256 校验保护，防止篡改。

5.4 违规行为

以下行为将被视为违规，组委会有权取消相关队伍的比赛资格：

1. 直接修改评分服务代码或判定逻辑
2. 篡改 ORCA 仿真环境中的物理参数
3. 使用非正常手段干扰其他队伍的比赛
4. 抄袭或盗用其他队伍的核心算法代码
5. 反编译或逆向工程评分服务二进制文件以获取评分规则
6. 修改 `tasks.yaml` 或 `robots.yaml` 配置文件绕过哈希校验

第 6 章

大赛评分规则与机制

6.1 三层判定架构

评分系统采用三层判定架构，从底层环境原语到高层声明式配置，实现灵活可扩展的评分判定：

```
Layer 3: YAML 解析层（condition_parser.py）
  tasks.yaml → Condition 判定树
  声明式配置，无需编写 Python 判定代码
  支持 ${robot.xxx} 占位符替换，实现机器人参数化配置

Layer 2: 谓词组合层（conditions.py）
  And / Or / Not / Threshold / Duration / WholeStep / SmoothCurve
  逻辑组合与时间约束

Layer 1: 环境原语层（primitives.py）
  point_in_region / site_distance / is_grasped / base_velocity_norm /
  collision_count / target_button_region / ctx_value / manual
  通过 gRPC 查询 ORCA 仿真状态
```

6.2 评分公式

步骤得分

```
step_score = step.success ? step.score : 0
```

任务得分率（归一化到 0-1）

```
attempt_score = sum(成功步骤分值) / sum(所有步骤分值)
```

加权总分（满分 100）

```
team_total = 任务1得分率 × 20 + 任务2得分率 × 40 + 任务3得分率 × 40
```

示例

假设某队伍完成情况如下：

任务	成功步骤	总步骤	得分率	加权分
任务一	4/5 (20分中得16分)	5	0.80	16.0
任务二	3/4 (40分中得30分)	4	0.75	30.0
任务三	5/6 (40分中得33分)	6	0.825	33.0
总分				**79.0**

6.3 判定原语详解

位置/区域类原语

原语	语法	用途	判定方式
`point_in_region`	`{point, center, radius, point_body, center_body, point_body_substr, center_body_substr}`	点进入圆形区域	XY 平面欧氏距离
`site_distance`	`[site_a, site_b]` + `le` / `ge`	两点距离判定	三维欧氏距离
`body_in_region`	复用 `point_in_region`	body 是否在某区域内	先查 site 再查 body
`site_proximity`	`{site_a, site_b, threshold}`	两 site 末端距离 <= threshold	默认 threshold=0.03m

接触/碰撞类原语

原语	语法	用途	判定方式
`is_grasped`	`{object, gripper, finger_a, finger_b, closed_threshold, proximity}`	抓取状态判定	夹爪闭合 + 物体接近
`collision_count`	`{body, body_name, body_substr, exclude_bodies}` + `le`	碰撞计数	query_contact_simple
`contact_between`	`{body_a, body_b}`	两个 body 是否存在接触	geom→body 映射

关节/运动类原语

原语	语法	用途	判定方式
`base_velocity_norm`	`{base_body, base_body_substr, base_site}` + `ge` / `le`	基座速度模长	多种速度查询接口回退
`joint_qpos`	`{joint_name, index}`	关节 qpos 值	query_joint_qpos
`gripper_width`	`{finger_a, finger_b}`	夹爪开合宽度	两关节 qpos 差绝对值

动态/上下文类原语

原语	语法	用途	说明

`target_button_region`	`{ee_site, radius}`	动态按钮区域判定	根据 `ctx.target_color` 选择对应按钮
`ctx_value`	`{key}`	从步骤上下文读取自定义值	队伍通过 `set_extra` 喂入
`manual`	`{}`	手动判定	队伍通过 `set_success` 控制

组合算子

算子	语法	用途
`all`	`[sub1, sub2, ...]`	AND 组合（全部条件满足）
`any`	`[sub1, sub2, ...]`	OR 组合（任一条件满足）
`not`	`<sub-block>`	NOT 取反
`duration`	`{predicate, min_s}`	持续成立 N 秒
`smooth_curve`	`{primitive, args, max_jerk}`	曲线平滑度（jerk RMS）
`whole_step`	`true`	全程性判定（20Hz 采样，每帧调用 observe_frame）

原语使用示例

YAML 声明式（在 `tasks.yaml` 中）：

```
# point_in_region 三种模式
point_in_region:
  point: robot_base              # site 模式
  center: Static_Location_A_site
  radius: 0.6
  point_body: ${robot.base_body} # body 全名模式

point_in_region:
  point: ${robot.ee_site}        # 占位符替换
  center: Group_Static_ElectricalCabinet_button01_site
  radius: 0.10

# is_grasped（YAML 友好字段名）
is_grasped:
  object: Group_Interactive_TestPencil_task_testpencil_testpoint_site
  gripper: ${robot.ee_site}
  finger_a: ${robot.finger_a}
  finger_b: ${robot.finger_b}
```

Python SDK 调用（在推理代码中）：

```
from orca_competition.primitives import (
    point_in_region, site_distance, is_grasped,
    base_velocity_norm, collision_count, target_button_region, manual, ctx_value
)

# 区域判定
cond = point_in_region("ee_site", "button01_site", 0.10,
                       point_body="g1_omnipicker_ee_center_site_l")

# 距离判定（链式阈值）
cond = site_distance("pen_tip", "panel").le(0.02)

# 抓取判定
cond = is_grasped("tool_body", "gripper_site",
                  "finger_a_joint", "finger_b_joint")

# 速度判定
cond = base_velocity_norm(base_body="g1_omnipicker_body_link1").ge(0.05)

# 碰撞判定
cond = collision_count("arm", body="body_link1").le(0)

# 动态按钮区域
cond = target_button_region(ee_site="ee_site", radius=0.10)

# 手动判定
cond = manual()

# 上下文值
cond = ctx_value("voltage").ge(220).for_duration(2.0)
```

6.4 诊断信息解读

每个 `end_step()` 返回的 `diagnosis` 字段包含详细判定信息：

```
{
    "passed": false,
    "reason": "末端距离按钮 0.852m > 0.10m",
    "actual": 0.852,
    "expected": "<=0.10"
}
```

字段	含义
`passed`	是否通过判定

`reason`	失败原因的人类可读描述
`actual`	实际测量值（距离/速度/碰撞数等）
`expected`	期望阈值

复合条件诊断：对于 `all(...)` 组合条件，`reason` 会显示第一个失败的子条件。例如工具归位步骤可能显示：

- `"工具未在工具箱区域内: 0.52m > 0.30m"` — 位置条件不满足
- `"工具仍被抓取中"` — 释放条件不满足
- `"工具在区域内持续时间不足: 0.3s < 1.0s"` — 稳定条件不满足

6.5 排行榜计算

```
avg_score = 各任务最佳得分率 × 任务权重之和（0-100）
best_score = 各任务最佳得分率 × 任务权重之和（0-100）
```

排行榜按 **avg_score 降序排列**。如果队伍多次尝试同一任务，系统取各任务的最佳得分率计算。

6.6 防作弊机制

6.6.1 二进制分发

评分服务（`scorer_service`）通过 PyInstaller 打包为独立可执行文件，参赛队伍无法直接查看或修改评分逻辑源代码。`ScorerClient` 优先启动二进制文件，仅在二进制不可用时回退到源码模式。

6.6.2 SHA-256 配置文件哈希校验

打包时生成 `configs/config.hash`，记录 `tasks.yaml` 和 `robots.yaml` 的 SHA-256 哈希值。评分服务启动时自动校验：

- **默认模式**：校验失败打印警告，服务仍可运行（允许使用自定义配置进行本地调试）
- **严格模式** `--strict-config`：校验失败直接退出，用于正式比赛环境

6.6.3 开发者约束

- 开发者文档中不包含修改评分规则逻辑的内容
- 评分规则文件（`tasks.yaml`、`robots.yaml`）的修改会被哈希校验检测到

- 正式比赛时管理员将使用 `--strict-config` 模式启动评分服务

第 7 章

大赛开发指南

7.1 环境准备

系统要求

- Python 3.10+
- 操作系统：Windows / Linux / macOS
- 需要能运行 ORCA 仿真环境
- 代码目录：`competition_scoring_client/`（本地开发端）

依赖安装

```
# 进入本地开发端目录
cd OrcaManipulation/src/examples/competition_scoring_client

# 安装 Python 依赖
pip install fastapi uvicorn pyyaml pydantic grpcio grpcio-tools

# 验证安装
python -c "import fastapi, uvicorn, yaml, pydantic, grpc; print('OK')"
```

7.2 ScorerClient SDK 详解

ScorerClient 是参赛队伍与评分服务交互的唯一接口，提供 **2 种评分模式** + 4 个核心方法 + 3 个辅助方法。所有方法通过 HTTP 与本地评分服务通信。

导入与初始化

```
# 推荐的新导入方式
from orca_scorer_client import ScorerClient

# 兼容的旧导入方式（功能相同）
from scorer_client import ScorerClient

# 初始化（必传 team_id 和 team_token）
# 评分服务会自动启动，无需手动预先启动
scorer = ScorerClient(
    team_id="team_003",           # 必传：队伍 ID，由管理员分配
    team_token="token_team_003", # 必传：队伍 Token，由管理员分配
)

# 可选参数（均使用默认值即可，无需修改）
scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
    server_url="http://8.159.130.194/",  # 中央服务器地址（默认值）
    base_url="http://127.0.0.1:9000",    # 本地评分服务地址（默认值）
    orca_addr="localhost:50051",         # ORCA gRPC 地址（默认值）
    auto_start=True,                     # 自动启动评分服务（默认 True）
    timeout=10.0,                        # HTTP 请求超时（默认 5.0 秒）
)
```

> **重要变更**：ScorerClient 现在**自动启动本地评分服务**，无需手动预先启动。初始化时传入 `team_id` 和 `team_token` 即可，退出时自动清理子进程。详见 10.3 步骤三。

两种评分模式

模式	API 调用	适用场景	说明
自动模式（推荐）	`start_attempt()` + `finish()`	所有任务	评分服务自动 20Hz 采样，finish 时回溯所有帧自动判定每个步骤
手动模式（兼容）	`start_attempt()` + `begin_step()` / `end_step()` + `finish()`	需要精确控制步骤边界的场景	逐步骤手动标记开始/结束，评分服务在每个 `end_step()` 时判定

自动模式（推荐）

仅需 **2 个 API 调用**，零侵入：

```
# 开始任务（评分服务自动开始 20Hz 后台采样）
info = scorer.start_attempt("task2_button_press")

# ... 你的机器人控制代码，完全不变 ...

# 结束任务（评分服务自动回溯所有帧，判定每个步骤）
summary = scorer.finish()
print(f"总分: {summary['score']}/{summary['max_score']}")
```

手动模式（兼容）

需要 **4 个 API 调用**，逐步骤手动标记：

1. `start_attempt(task_id: str) -> dict`

开始一次任务尝试。返回任务元信息和步骤列表。

```
info = scorer.start_attempt("task2_button_press")
# 返回示例:
# {
#     "attempt_id": "abc123",
#     "task_name": "语言指令按钮操作任务",
#     "total_steps": 4,
#     "steps": [
#         {"step_id": "press_red_button", "name": "红色按钮...", "score": 10},
#         {"step_id": "press_green_button", "name": "绿色按钮...", "score": 10},
#         ...
#     ]
# }
```

2. `begin_step(step_id: str) -> dict`

开始一个步骤。

```
result = scorer.begin_step("press_red_button")
# 返回: {"step_id": "press_red_button", "name": "...", "score": 10, "status":
"started"}
```

3. `end_step() -> dict`

结束当前步骤。评分服务自动通过 gRPC 查询 ORCA 仿真状态进行判定，返回判定结果和诊断信息。

```
result = scorer.end_step()
# 返回示例:
# {
#     "step_id": "press_red_button",
#     "success": True,
#     "score": 10,
#     "diagnosis": {
#         "passed": True,
#         "reason": "末端距离按钮 0.052m <= 0.10m",
#         "actual": 0.052,
#         "expected": "<=0.10"
#     }
# }
```

4. `finish() -> dict`

结束任务尝试，返回汇总信息。

```
summary = scorer.finish()
# 返回示例:
# {
#     "attempt_id": "abc123",
#     "score": 30,
#     "max_score": 40,
#     "success_rate": 0.75
# }
```

辅助方法

方法	说明
`set_extra(key, value)`	设置步骤额外信息（如 `target_color`、`voltage` 等上下文值）
`status()`	获取当前评分服务状态（ORCA 连接、进度等）
`history()`	获取历史尝试记录列表

错误处理

ScorerClient 在 HTTP 错误时会返回包含 `error` 字段的字典，而非抛出异常：

```
result = scorer.end_step()
if "error" in result:
    print(f"评分服务连接失败: {result['error']}")
    # 常见原因：评分服务未启动、端口错误、网络不通
```

7.3 完整示例代码

示例一：自动模式（推荐，仅 2 个 API 调用）

```
"""自动评分模式 — 适用于所有三个任务，仅需 2 个 API 调用"""
from scorer_client import ScorerClient

# 初始化（自动启动评分服务）
scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

# ==========================================
# 任务一：移动巡检
# ==========================================
info = scorer.start_attempt("task1_inspection")
print(f"任务: {info['task_name']}")

# ... 你的机器人导航代码（如 WaypointNavController） ...

summary = scorer.finish()
print(f"任务一总分: {summary['score']}/{summary['max_score']}")
for step in summary.get("steps", []):
    status = "PASS" if step["success"] else "FAIL"
    print(f"  {step['step_id']}: {status} | {step.get('diagnosis', {}).get('reason',
'')}")

# ==========================================
# 任务二：按钮操作
# ==========================================
info = scorer.start_attempt("task2_button_press")
print(f"任务: {info['task_name']}")

# ... 你的机器人按钮操作代码 ...

summary = scorer.finish()
print(f"任务二总分: {summary['score']}/{summary['max_score']}")

# ==========================================
# 任务三：工具归纳
# ==========================================
info = scorer.start_attempt("task3_tool_storage")
print(f"任务: {info['task_name']}")

# ... 你的机器人工具操作代码 ...

summary = scorer.finish()
print(f"任务三总分: {summary['score']}/{summary['max_score']}")
```

示例二：手动模式（逐步骤标记，兼容方式）

```
"""手动评分模式 — 逐步骤标记开始/结束"""
from scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

# 开始任务二
info = scorer.start_attempt("task2_button_press")
print(f"任务: {info['task_name']}, 共 {info['total_steps']} 步")

# 遍历每个步骤
for step_info in info["steps"]:
    step_id = step_info["step_id"]
    scorer.begin_step(step_id)

    # ==========================================
    # 你的推理和控制代码
    # 例如: 解析自然语言指令 → 识别目标颜色 →
    #       规划路径 → 控制机器人末端移动到按钮位置
    # ==========================================

    result = scorer.end_step()
    status = "PASS" if result["success"] else "FAIL"
    diag = result.get("diagnosis", {})
    print(f"  {step_id}: {status} | {diag.get('reason', '')}")

# 结束任务
summary = scorer.finish()
print(f"\n总分: {summary['score']}/{summary['max_score']}")
```

示例三：手动模式 — 任务一（移动巡检）

```
from scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

info = scorer.start_attempt("task1_inspection")

for step_info in info["steps"]:
    step_id = step_info["step_id"]
    scorer.begin_step(step_id)

    # ========================================
    # 你的移动导航代码
    # 例如：规划路径 → 发送移动指令 →
    #       检测到达检查点 → 继续移动到下一目标
    # ========================================

    result = scorer.end_step()
    print(f"  {step_id}: {result['success']}")

summary = scorer.finish()
print(f"总分: {summary['score']}/{summary['max_score']}")
```

示例四：手动模式 — 任务三（工具归纳）

```
from scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

info = scorer.start_attempt("task3_tool_storage")

# 工具列表（按步骤顺序）
tools = ["reach_tool_table", "pick_voltage_pen", "pick_screwdriver",
         "pick_wrench", "pick_flashlight", "pick_electrician_knife"]

for step_id in tools:
    scorer.begin_step(step_id)

    # ========================================
    # 你的工具操作代码
    # 例如：到达工具箱 → 拾取工具 → 移动至工具箱 →
    #       释放工具 → 等待稳定 → 下一件工具
    # ========================================

    result = scorer.end_step()
    print(f"  {step_id}: {result['success']}")

summary = scorer.finish()
print(f"总分: {summary['score']}/{summary['max_score']}")
```

7.4 中央服务器 API 参考

中央服务器提供完整的 REST API 供参赛队伍上报结果和查询数据。所有 API 需要 JWT 鉴权。

认证接口

方法	路径	说明
POST	`/api/auth/login`	统一登录（返回 JWT Token）
GET	`/api/auth/verify`	验证 Token 有效性

登录请求示例：

```
// 开发者登录
{
    "role": "team",
    "identifier": "team_001",
    "token": "your_team_token"
}
```

结果上报接口

方法	路径	说明
POST	`/api/v1/attempts`	创建尝试记录
POST	`/api/v1/attempts/{id}/steps`	上报步骤结果
POST	`/api/v1/attempts/{id}/finish`	结束尝试
POST	`/api/v1/attempts/{id}/summary`	上报汇总信息

查询接口

方法	路径	说明
GET	`/api/v1/teams/{id}`	本队详情（含统计和排名）
GET	`/api/v1/teams/{id}/attempts`	本队所有尝试
GET	`/api/v1/teams/{id}/step_stats`	本队各步骤成功率画像
GET	`/api/v1/leaderboard`	排行榜（team 只能看自己，返回 403）

文件上传接口

方法	路径	说明
POST	`/api/v1/teams/{id}/uploads`	上传文件（支持 code/video/doc，最大 200MB）
GET	`/api/v1/teams/{id}/uploads`	查询本队上传
GET	`/api/v1/uploads/{upload_id}/stream`	视频流媒体播放（HTTP Range 支持）

公开接口

方法	路径	说明
GET	`/api/guide`	大赛指南（Markdown 内容）
GET	`/api/dev-guide`	开发者接入指南
GET	`/api/v1/health`	健康检查
GET	`/api/v1/branding`	品牌配置（Logo、标题）
GET	`/api/v1/tasks`	任务列表
GET	`/api/v1/scores`	分数统计

管理接口（仅 admin）

方法	路径	说明
PUT	`/api/admin/guide-content`	编辑大赛指南
PUT	`/api/admin/dev-guide-content`	编辑开发者指南
PUT	`/api/admin/admin-docs-content`	编辑管理员文档
PUT	`/api/v1/admin/branding`	更新品牌配置
POST	`/api/v1/admin/branding/logo`	上传品牌 Logo
PUT	`/api/v1/admin/teams`	新增队伍
DELETE	`/api/v1/admin/teams/{id}`	删除队伍
PUT	`/api/v1/admin/judges`	新增评委
DELETE	`/api/v1/admin/judges/{id}`	删除评委
POST	`/api/v1/admin/reset`	数据重置

鉴权机制

中央服务器采用自实现 JWT（HS256）鉴权，支持三种角色：

- **公开路径**：`/api/guide`、`/api/v1/health`、`/api/v1/branding` 等无需鉴权

- **上报路径**：`/api/v1/attempts`、`/api/v1/teams/` 支持 `X-Team-Token` 或 JWT Bearer Token 鉴权
- **管理路径**：需要 JWT Bearer Token + admin 角色

第 8 章

任务实现方案详解

本章提供三个任务的详细实现方案，包括机器人驱动方式、导航控制、末端操作和工具抓取放置的完整技术细节。

8.1 通用驱动方式

无论使用哪个任务，当前已验证的 ORCA 仿真机器人驱动方式均为：

```
DataCollection env → 直接设置 free_joint / 关节 qpos/qvel → env.step() → env.render()
```

核心步骤：

1. **初始化环境**：通过 `SceneManager` 加载场景，通过 `DataCollectionManager` 创建数据采集环境
2. **装配控制器**：根据任务需求装配对应的控制器（如 `ControllerSteeringDrive` 用于底盘移动，`ControllerArm` 用于臂部操作）
3. **推理循环**：每帧调用 `run_controllers()` → `env.step(action)` → `env.render()`
4. **评分交互**：在推理循环中通过 `ScorerClient` 的 `begin_step()` / `end_step()` 与评分服务交互

关键参数：

参数	值	说明
仿真步长	0.005s	MuJoCo 物理步长
控制帧率	40Hz（frame_skip=5）	每 5 个物理步执行一次控制
评分采样率	20Hz	评分服务后台采样频率

8.2 任务一实现方案：移动巡检

8.2.1 导航控制器架构

任务一使用 **闭环路点导航控制器**（`WaypointNavController`），位于 `nav_task1_test/nav_controller.py`（446 行），基于阿克曼转向模型：

```
路点序列 A → B → C → 到达判定 → 切换下一路点
```

位姿估计： - 位置：优先从 free_joint qpos 读取（`query_joint_qpos`），其次从 body xpos 读取 - 朝向：从 free_joint 四元数提取 yaw 角（`query_joint_qpos` 的 3:7 分量） - 速度：优先从 site 速度读取（`query_site_xvalp_xvalr`），其次 body cvel，最后位置差分

控制律（纯前向追踪）：

```
转向角 = steer_kp × heading_error（限制在 ±max_steer_input）
油门   = throttle（固定值，靠近路点时减速）
```

- 朝向误差 > 60° 时减速 + 大转向
- 朝向误差 < 0.05 rad（死区）时直行
- 倒车模式（带滞回）：进入 > 97°，退出 < 63°
- 到达判定：距离 <= 0.5m（arrival_radius）

自由关节直接操控（绕过轮子物理问题）：

```
move_speed = abs(throttle) * 3.0          # m/s
dt = 0.025                                 # 控制周期
max_turn_rate = 1.5                        # rad/s

# 每帧更新 free_joint 位姿
heading += steering * max_turn_rate * dt
pos_x += move_speed * cos(heading) * dt
pos_y += move_speed * sin(heading) * dt
qpos[0:3] = [pos_x, pos_y, locked_z]      # 锁定 Z 轴
qpos[3:7] = quat_from_yaw(heading)         # 绕 Z 轴旋转
qvel[0:3] = [vx, vy, 0]                    # 设置线速度用于 speed 判定
```

8.2.2 路点配置（nav_task1.yaml）

```yaml
robot_profile: "g1_omnipicker"
waypoints:
  - name: "检查点 A"
    site: "Static_Location_A_site"
    radius: 0.5
    description: "路径经过点 A"
  - name: "检查点 B"
    site: "Static_Location_B_site"
    radius: 0.5
    description: "路径经过点 B"
  - name: "设备柜 C"
    site: "Static_Location_C_site"
    radius: 0.5
    description: "最终到达点"

nav_params:
  throttle: 0.6          # 油门（0-1）
  heading_deadzone_rad: 0.05
  arrival_radius_m: 0.5
  steer_kp: 1.5          # 转向 P 增益
  max_steer_input: 1.0

control_params:
  max_steps: 1500        # 最大步数
  max_duration: 120      # 最大时长（秒）
```

8.2.3 步骤与路点对应关系

步骤	对应路点	判定方式
`start_move`	-	base 速度 >= 0.05 m/s 持续 1s（自动判定，无需特殊导航）
`pass_checkpoint_a`	路点 0（A）	base 进入半径 0.6m 区域
`pass_checkpoint_b`	路点 1（B）	base 进入半径 0.6m 区域
`arrive_panel`	路点 2（C）	base 进入半径 0.6m 区域
`smooth_motion`	全程	速度 jerk <= 2.0 + 全程无碰撞（全程性判定）

8.2.4 完整代码示例

```
"""任务一：移动巡检 — 完整实现示例"""
from scorer_client import ScorerClient
from nav_task1_test.nav_controller import WaypointNavController, NavWaypoint
import yaml, time

# 1. 加载配置
with open("nav_task1_test/nav_task1.yaml") as f:
    config = yaml.safe_load(f)

# 2. 初始化 ORCA 环境（省略详细初始化代码，参考 run_nav_task1.py）
env = init_orca_env()  # SceneManager + DataCollectionManager

# 3. 初始化评分客户端（自动启动评分服务）
scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

# 4. 构建路点列表
waypoints = [NavWaypoint(wp["site"], wp["radius"], wp["description"])
             for wp in config["waypoints"]]

# 5. 初始化导航控制器
nav = WaypointNavController(
    waypoints=waypoints,
    base_body=config["robot"]["base_body"],
    free_joint=config["robot"]["free_joint"],
    throttle=config["nav_params"]["throttle"],
    heading_deadzone_rad=config["nav_params"]["heading_deadzone_rad"],
    arrival_radius_m=config["nav_params"]["arrival_radius_m"],
    steer_kp=config["nav_params"]["steer_kp"],
)

# 6. 开始任务
info = scorer.start_attempt("task1_inspection")
step_idx = 0

for step_info in info["steps"]:
    step_id = step_info["step_id"]
    scorer.begin_step(step_id)
    print(f"执行步骤: {step_info['name']} ({step_info['score']}分)")
```

```
    # 导航控制循环
    for _ in range(config["control_params"]["max_steps"]):
        state = nav.update(env)  # 计算 throttle/steering

        # 应用控制到仿真
        apply_steering_drive(env, state.throttle, state.steering)
        env.step()
        env.render()

        # 到达路点则跳出
        if state.arrived and step_idx < len(waypoints):
            break

    result = scorer.end_step()
    print(f"  结果: {'PASS' if result['success'] else 'FAIL'}")
    if not result['success']:
        print(f"  诊断: {result.get('diagnosis', {}).get('reason', '')}")
    step_idx += 1

summary = scorer.finish()
print(f"任务一总分: {summary['score']}/{summary['max_score']}")
```

已验证结果：使用 G1 OmniPicker + 自由关节直接操控，可取得满分 **20/20**。

8.3 任务二实现方案：按钮操作

8.3.1 末端执行器控制

任务二需要控制机器人末端执行器（ee_site）移动到指定按钮位置。G1 OmniPicker 使用双臂 OSC（操作空间控制）控制器：

臂部配置（来自 `g1_omnipicker_conf.py`）：

参数	左臂	右臂
关节数	7	7
中性位置	`[-1.42, 0.88, 1.54, -1.48, 0, 0, 0]`	`[1.42, -0.88, -1.54, 1.48, 0, 0, 0]`
末端 site	`ee_center_site_l`	`ee_center_site_r`
电机范围	`(-60, 60)` / `(-30, 30)`	`(-60, 60)` / `(-30, 30)`

控制器创建：

```
from controllers.controllers import create_arm_osc_controller

# 创建左臂 OSC 控制器
arm_ctrl = create_arm_osc_controller(
    env, arm_config, "left_arm",
    init_ctrl=[0]*7, base_body="g1_omnipicker_body_link1"
)
```

8.3.2 按钮识别与定位

按钮在 ORCA 场景中的位置固定，可通过 site 名称直接查询：

```
# 查询按钮位置
button_sites = {
    "red": "Group_Static_ElectricalCabinet_button01_site",
    "green": "Group_Static_ElectricalCabinet_button02_site",
    "blue": "Group_Static_ElectricalCabinet_button03_site",
    "yellow": "Group_Static_ElectricalCabinet_button04_site",
}

# 通过 ORCA 查询按钮 site 位置
button_pos =
env.query_site_pos_and_mat(["Group_Static_ElectricalCabinet_button01_site"])
target_pos = button_pos[0][:3, 3]  # 从 4x4 矩阵提取位置
```

自然语言指令解析：参赛队伍需要自行实现自然语言指令解析模块，从指令中提取目标颜色（红/绿/蓝/黄），然后选择对应的按钮 site。

8.3.3 完整代码示例

```
"""任务二：按钮操作 — 完整实现示例"""
from scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

# 按钮 site 映射
BUTTON_MAP = {
    "red": "Group_Static_ElectricalCabinet_button01_site",
    "green": "Group_Static_ElectricalCabinet_button02_site",
    "blue": "Group_Static_ElectricalCabinet_button03_site",
    "yellow": "Group_Static_ElectricalCabinet_button04_site",
}

# 开始任务
info = scorer.start_attempt("task2_button_press")

for step in info["steps"]:
    step_id = step["step_id"]
    color = step_id.replace("press_", "").replace("_button", "")
    scorer.begin_step(step_id)

    # 1. 查询目标按钮位置
    button_site = BUTTON_MAP.get(color)
    if button_site:
        result = env.query_site_pos_and_mat([button_site])
        target_pos = np.array(result[0][:3, 3])
    else:
        target_pos = np.array([0, 0, 0])  # fallback

    # 2. 控制末端执行器移动到按钮位置
    #    使用 OSC 控制器设置目标位姿
    arm_ctrl.update_goal(target_pos - ee_pos, target_quat)
    for _ in range(200):  # 最多 200 步
        ctrl = arm_ctrl.run_controller()
        env.step(ctrl)
        env.render()

        # 检查是否到达（距离 < 0.05m 可提前结束）
        ee_pos = get_ee_position(env, "g1_omnipicker_ee_center_site_l")
```

```
        if np.linalg.norm(ee_pos - target_pos) < 0.05:
            break

    result = scorer.end_step()
    status = "PASS" if result["success"] else "FAIL"
    print(f"  {step_id}: {status}")

summary = scorer.finish()
print(f"任务二总分: {summary['score']}/{summary['max_score']}")
```

已知限制：ORCA 场景中可能缺少 `button03_site`（蓝色按钮），该步骤预期会失败。请以实际 ORCA 场景中的 site 列表为准。

8.4 任务三实现方案：工具归纳

8.4.1 抓取与放置流程

任务三的每件工具操作遵循统一的"拾取→归位"流程：

```
1. 到达作业位姿 → 2. 移动末端到工具 → 3. 闭合夹爪抓取
→ 4. 移动工具到工具箱 → 5. 打开夹爪释放 → 6. 等待稳定
```

夹爪控制（2F85 夹爪）：

参数	左夹爪	右夹爪
执行器	`idx39_gripper_l_inner_joint2_pctrl` + `idx49_gripper_l_outer_joint2_pctrl`	`idx79_gripper_r_inner_joint2_pctrl` + `idx89_gripper_r_outer_joint2_pctrl`
范围	`(-1.0, 2)`	`(-1.0, 2)`
闭合值	~2.0（夹紧）	~2.0（夹紧）
打开值	~-1.0（释放）	~-1.0（释放）

工具 site 映射：

```
TOOLS = {
    "pick_voltage_pen":
"Group_Interactive_TestPencil_task_testpencil_testpoint_site",
    "pick_screwdriver": "Group_Interactive_Screwdriver_task_screwdriver_site",
    "pick_wrench": "Group_Interactive_Spanner_task_spanner_site",
    "pick_flashlight": "Group_Interactive_Flashlight_task_flashlight_site",
    "pick_electrician_knife":
"Group_Interactive_ElectriciansKnife01_task_electriciansknife01_site",
}
TOOLBOX = "Group_Interactive_ToolBox_base_site"
```

8.4.2 工具归位三条件（必须同时满足）

每件工具归位成功的判定基于三个条件（YAML 中的 `all` 组合）：

1. **位置条件**：工具 site 在工具箱基座区域（`Group_Interactive_ToolBox_base_site`）半径 **0.3m** 内
2. **释放条件**：工具**未被**夹爪抓取（`NOT is_grasped`）
3. **稳定条件**：工具在工具箱区域内**持续 1 秒以上**

这要求参赛队伍在释放工具后，必须等待足够的时间（>= 1 秒）让工具稳定在工具箱内，才能调用 `end_step()` 触发判定。

8.4.3 完整代码示例

```
"""任务三：工具归纳 — 完整实现示例"""
from scorer_client import ScorerClient
import numpy as np
import time

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

TOOLS = {
    "pick_voltage_pen":
"Group_Interactive_TestPencil_task_testpencil_testpoint_site",
    "pick_screwdriver": "Group_Interactive_Screwdriver_task_screwdriver_site",
    "pick_wrench": "Group_Interactive_Spanner_task_spanner_site",
    "pick_flashlight": "Group_Interactive_Flashlight_task_flashlight_site",
    "pick_electrician_knife":
"Group_Interactive_ElectriciansKnife01_task_electriciansknife01_site",
}
TOOLBOX_SITE = "Group_Interactive_ToolBox_base_site"

info = scorer.start_attempt("task3_tool_storage")

for step in info["steps"]:
    step_id = step["step_id"]
    scorer.begin_step(step_id)

    if step_id == "reach_tool_table":
        # 步骤 1：到达作业位姿（移动 base 到工具箱附近）
        toolbox_pos = get_site_pos(env, TOOLBOX_SITE)
        navigate_base_to(env, toolbox_pos, radius=0.5)
    else:
        # 步骤 2-6：拾取工具并归位
        tool_site = TOOLS.get(step_id)
        toolbox_pos = get_site_pos(env, TOOLBOX_SITE)

        # 1) 移动到工具位置
        tool_pos = get_site_pos(env, tool_site)
        move_ee_to(env, tool_pos + np.array([0, 0, 0.1]))  # 在工具上方悬停

        # 2) 下降并抓取
        move_ee_to(env, tool_pos)
        close_gripper(env)  # 闭合夹爪
        time.sleep(0.5)
```

```
        # 3) 抬起工具
        move_ee_to(env, tool_pos + np.array([0, 0, 0.2]))

        # 4) 移动到工具箱上方
        target = toolbox_pos + np.array([0, 0, 0.15])
        move_ee_to(env, target)

        # 5) 释放工具
        open_gripper(env)  # 打开夹爪
        time.sleep(1.5)    # 等待工具稳定（>= 1 秒）

    result = scorer.end_step()
    status = "PASS" if result["success"] else "FAIL"
    diag = result.get("diagnosis", {})
    print(f"  {step_id}: {status} | {diag.get('reason', '')}")

summary = scorer.finish()
print(f"任务三总分: {summary['score']}/{summary['max_score']}")
```

注意事项： - 释放工具后必须等待 >= 1 秒，否则稳定条件（duration 判定）不满足 - 工具必须在工具箱区域内（距离 <= 0.3m），且夹爪完全打开 - 当前 Task3 步骤判定基本失败，主要原因是工具尚未被有效放置到工具箱区域内

8.5 模型推理集成

参赛队伍可以使用训练好的模型（如 OpenPI、扩散策略等）进行推理控制。以下是在推理循环中集成评分 SDK 的示例骨架：

```
"""模型推理 + 评分 SDK 集成骨架"""
from scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

# 状态/动作布局（18 维，OpenPI 格式）
# [0:3]   l_pos_b     左臂末端位置（base 系）
# [3:7]   l_quat_b    左臂末端四元数
# [7:10]  r_pos_b     右臂末端位置
# [10:14] r_quat_b    右臂末端四元数
# [14]    l_grip_inner 左夹爪内侧归一化 [0,1]
# [15]    l_grip_outer 左夹爪外侧归一化 [0,1]
# [16]    r_grip_inner 右夹爪内侧归一化 [0,1]
# [17]    r_grip_outer 右夹爪外侧归一化 [0,1]

info = scorer.start_attempt("task2_button_press")

for step in info["steps"]:
    scorer.begin_step(step["step_id"])

    # 推理循环
    for _ in range(max_steps):
        # 1. 构建观测（state + images + prompt）
        state = build_observation(env)
        images = env.get_camera_images(["cam_head", "cam_wrist_l"])

        # 2. 模型推理（通过 WebSocket 连接策略服务器）
        action = infer_action(state, images)

        # 3. 执行动作
        env.step(action)
        env.render()

    result = scorer.end_step()

summary = scorer.finish()
```

相机配置（G1 OmniPicker）： - `cam_head`：头部相机 - `cam_wrist_l`：左手腕相机 - `cam_wrist_r`：右手腕相机

夹爪反归一化公式（OpenPI 格式）：

```
grip_val = norm_val * 3 - 1  # 将 [0, 1] 映射到 (-1, 2)
```

第 9 章

ORCA 仿真平台 API 详细参考

本章详细介绍 ORCA 仿真平台的核心 API，包括环境初始化、观测构建、动作定义、控制器体系、机器人配置、场景管理、数据采集和模型推理部署。

9.1 环境定义与初始化

9.1.1 DataCollectionEnv

ORCA 仿真环境基于 `OrcaGymLocalEnv` 实现，核心入口为 `DataCollectionEnv`（位于 `src/envs/dataCollection/dataCollection_env.py`）：

```
class DataCollectionEnv(OrcaGymLocalEnv):
    def init_env(self):
        """初始化仿真环境"""
        self.initialize_simulation()

    def step(self, action):
        """推进物理仿真
        Args:
            action: 控制信号，维度 = model.nu，范围 [-1, 1]
        """
        self.do_simulation(self.ctrl, self.frame_skip)

    def reset_model(self):
        """重置到默认关节位置"""
        ...

    def _get_obs(self):
        """构建观测（通过 obs_callback 自定义）"""
        return self.obs_callback(self)
```

9.1.2 环境初始化流程

标准初始化流程如下：

```
# 1. 创建场景管理器
from scene.scene_manager import SceneManager
scene_manager = SceneManager(grpc_addr="localhost:50051", config=scene_config)

# 2. 创建数据采集管理器
from dataCollectionManager.data_collection_manager import DataCollectionManager
dc_manager = DataCollectionManager(
    scene_manager=scene_manager,
    config=dc_config,
    mode="TELECONTROL"  # 或 "AUGMENTATION"
)

# 3. 获取环境实例
env = dc_manager.env  # DataCollectionEnv 实例
```

9.1.3 关键环境接口

接口	说明	返回值
`env.step(action)`	推进物理仿真 N 帧	observation, reward, terminated, truncated, info
`env.reset()`	重置环境到初始状态	observation, info
`env.render()`	渲染当前帧（在 ORCA Viewer 中显示）	-
`env.model`	MuJoCo 模型对象	可访问 body/site/joint 等属性
`env.data`	MuJoCo 数据对象	可访问 qpos/qvel 等状态

9.2 观测构建

9.2.1 Site 位置查询

```
# 查询 site 的 4x4 位姿矩阵（位置 + 旋转矩阵）
result = env.query_site_pos_and_mat(["site_name"])
pos = result[0][:3, 3]      # 世界坐标系位置 [x, y, z]
mat = result[0][:3, :3]      # 3x3 旋转矩阵
```

9.2.2 关节状态查询

```
# 查询关节位置
qpos = env.query_joint_qpos(["joint_name"])
# 查询关节速度
qvel = env.query_joint_qvel(["joint_name"])
```

9.2.3 Body 状态查询

```
# 查询 body 位置和姿态
xpos, xmat, xquat = env.get_body_xpos_xmat_xquat("body_name")
# xpos: [x, y, z] 世界坐标
# xmat: 3x3 旋转矩阵
# xquat: [w, x, y, z] 四元数

# 查询 body 速度（质心线速度 + 角速度）
cvel = env.query_body_cvel("body_name")  # [vx, vy, vz, wx, wy, wz]
```

9.2.4 Site 速度查询

```
# 查询 site 的线速度和角速度
xvalp, xvalr = env.query_site_xvalp_xvalr("site_name")
# xvalp: [vx, vy, vz] 线速度
# xvalr: [wx, wy, wz] 角速度
```

9.2.5 碰撞查询

```
# 查询 body 的碰撞接触对
contacts = env.query_contact_simple("body_name")
# 返回与 body_name 有接触的 geom 对列表

# 查询两个 body 之间是否存在接触
has_contact = env.check_contact_between("body_a", "body_b")
```

9.2.6 实体列表查询

```
# 列出场景中所有可用的 body/site/joint
bodies = env.list_available_bodies()   # body 名称列表
sites = env.list_available_sites()     # site 名称列表
joints = env.list_available_joints()   # joint 名称列表
```

9.2.7 相机图像获取

```
# 获取相机图像（需要在 env 配置中注册相机）
images = env.get_camera_images(["cam_head", "cam_wrist_l", "cam_wrist_r"])
# 返回字典：{"cam_head": np.ndarray, "cam_wrist_l": np.ndarray, ...}
```

9.3 动作定义与下发

9.3.1 动作空间

动作空间维度 = `model.nu`（MuJoCo 执行器数量），每个维度的范围为 `[-1, 1]`。对于 G1 OmniPicker，执行器包括：

执行器组	数量	范围	说明
左臂关节	7	`(-60,60)` / `(-30,30)`	7-DOF 臂部
右臂关节	7	`(-60,60)` / `(-30,30)`	7-DOF 臂部
左夹爪	2	`(-1, 2)`	内侧 + 外侧
右夹爪	2	`(-1, 2)`	内侧 + 外侧
底盘驱动	4	`(-30,30)` 速度 / `(-0.6,0.6)` 转向	2 轮驱动 + 2 轮转向

9.3.2 动作下发方式

```
# 方式一：直接下发动作值
action = np.zeros(env.model.nu)
# 设置具体关节值
action[arm_joint_indices] = arm_control_values
action[gripper_indices] = gripper_control_values
action[drive_indices] = drive_control_values
env.step(action)

# 方式二：通过控制器下发（推荐）
manager.run_controllers()  # 自动合并所有控制器的输出
env.step(merged_action)
```

9.3.3 自由关节直接操控（Task1 推荐方式）

对于移动导航任务，推荐直接操控自由关节（绕过轮子物理）：

```
# 获取自由关节索引
free_joint_qpos_addr = env.model.joint("g1_omnipicker_free_joint").qposadr
free_joint_qvel_addr = env.model.joint("g1_omnipicker_free_joint").dofadr

# 设置位置
env.data.qpos[free_joint_qpos_addr:free_joint_qpos_addr+3] = [x, y, z]
env.data.qpos[free_joint_qpos_addr+3:free_joint_qpos_addr+7] = quat  # 四元数

# 设置速度（评分系统通过此速度判定 base_velocity）
env.data.qvel[free_joint_qvel_addr:free_joint_qvel_addr+3] = [vx, vy, vz]

# 推进仿真
env.step(np.zeros(env.model.nu))
```

9.4 控制器详解

ORCA 提供多种控制器，通过工厂函数创建和管理。所有控制器继承自 `AbstractController`（位于 `src/controllers/abstract_controller.py`）。

9.4.1 控制器基类

```
class AbstractController:
    def __init__(self, env, ctrl_name, init_ctrl, base_body)
    def init_ctrl_index(self) -> list[int]       # 返回执行器索引列表
    def get_init_ctrl(self) -> dict[int, float]  # 返回初始控制值
    def reset(self)                              # 每个 episode 开始时调用
    def run_controller(self) -> dict[int, float]  # 必须实现：返回控制值字典
```

9.4.2 臂部控制器（ControllerArm — OSC/IK）

文件：`src/controllers/controller_arm.py`

```
from controllers.controllers import create_arm_osc_controller

# OSC 控制器（操作空间控制，推荐）
arm_ctrl = create_arm_osc_controller(
    env, arm_config, "left_arm",
    init_ctrl=[0]*7, base_body="g1_omnipicker_body_link1"
)

# 设置目标位姿（相对当前位置的偏移）
arm_ctrl.update_goal(relative_position, relative_quat)
# 或设置绝对目标位姿
arm_ctrl.set_absolute_goal(world_position, world_quat)

# 运行控制器
ctrl = arm_ctrl.run_controller()  # 返回 {joint_index: value, ...}
```

9.4.3 阿克曼转向驱动控制器（ControllerSteeringDrive）

文件： `src/controllers/controller_wheel_drive.py`

```
from controllers.controllers import create_steering_drive_controller

# 创建阿克曼转向控制器
drive_ctrl = create_steering_drive_controller(
    env, drive_config, "steering_drive",
    init_ctrl=[0, 0, 0, 0],  # [左轮速度, 右轮速度, 左轮转向, 右轮转向]
    base_body="g1_omnipicker_body_link1"
)

# 更新油门和转向
drive_ctrl.update_throttle(throttle)  # throttle: -1.0 ~ 1.0
drive_ctrl.update_steering(steering)  # steering: -1.0 ~ 1.0

# 阿克曼几何：转弯时内侧车轮转角 > 外侧车轮
# 转向采用指数映射：steer = (exp(k*|x|) - 1) / (exp(k) - 1) * max_steer_angle
```

9.4.4 夹爪控制器（2F85 Gripper）

```
from controllers.controllers import create_gripper_2f85_controller

# 创建 2F85 夹爪控制器
gripper_ctrl = create_gripper_2f85_controller(
    env, gripper_config, "left_gripper",
    init_ctrl=[-1.0, -1.0],  # 初始打开状态
    base_body="g1_omnipicker_body_link1"
)

# 闭合夹爪
gripper_ctrl.close()   # 设置目标值为 2.0（夹紧）
# 打开夹爪
gripper_ctrl.open()    # 设置目标值为 -1.0（释放）
```

9.4.5 控制器管理

```
# 通过 DataCollectionManager 管理所有控制器
manager = DataCollectionManager(...)

# 添加控制器
manager.add_controller("arm", arm_ctrl)
manager.add_controller("drive", drive_ctrl)
manager.add_controller("gripper", gripper_ctrl)

# 运行所有控制器（合并输出）
merged_action = manager.run_controllers()
env.step(merged_action)
```

9.5 机器人配置

9.5.1 G1 OmniPicker 完整配置（`src/conf/g1_omnipicker_conf.py`）

组件	参数	值
左臂	关节数	7
	中性位置	`[-1.42, 0.88, 1.54, -1.48, 0, 0, 0]`
	末端 site	`ee_center_site_l`

右臂	关节数	7
	中性位置	`[1.42, -0.88, -1.54, 1.48, 0, 0, 0]`
	末端 site	`ee_center_site_r`
左夹爪	执行器	`idx39_gripper_l_inner_joint2_pctrl` + `idx49_gripper_l_outer_joint2_pctrl`
	范围	`(-1.0, 2)`
右夹爪	执行器	`idx79_gripper_r_inner_joint2_pctrl` + `idx89_gripper_r_outer_joint2_pctrl`
	范围	`(-1.0, 2)`
底盘	驱动轮	`wheel_fl_joint_mctrl` + `wheel_fr_joint_mctrl`（速度范围 `(-30,30)`）
	转向轮	`wheel_fl_steer_joint_pctrl` + `wheel_fr_steer_joint_pctrl`（角度范围 `(-0.6,0.6)`）
	轴距	0.42m
	轮距	0.26m
基座	base_body	`body_link1`
	自由关节	`free_joint`

9.5.2 执行器分组

```
# 执行器组 ID（用于区分不同类型的控制）
motors_group = 0         # 电机类执行器
positions_group = 1      # 位置类执行器
```

9.6 场景管理

9.6.1 SceneManager（`src/scene/scene_manager.py`）

SceneManager 负责场景的加载、Actor 管理、随机化和配置校验。

```
from scene.scene_manager import SceneManager

# 初始化场景管理器
scene_manager = SceneManager(
    grpc_addr="localhost:50051",  # ORCA gRPC 地址
    config=scene_config,          # 场景配置字典
    env=orca_env,                 # OrcaGymLocalEnv 实例
)

# 核心方法
scene_manager.spawn_scene()       # 发布场景、生成 actors、设置灯光材质
scene_manager.spawn_actors()      # 生成可交互物体（瓶子、工具等）
scene_manager.update_actor_qpos() # 随机化或恢复 actor 位置
scene_manager.publish_scene()     # 发布场景并等待初始化
```

9.6.2 Actor 配置（YAML 格式）

```
actor:
  names: ["bottle_red", "bottle_blue"]
  spawnable: ["assets/items/bottle_red", "assets/items/bottle_blue"]
  joints: ["bottle_red_joint", "bottle_blue_joint"]
  joints_dof: [6, 6]                # 6-dof: 自由物体, 3-dof: 平移, 1-dof: 铰链
  random:
    qpos: true                      # 随机化位置
    material: false                 # 随机化材质
    nums: [3, 5]                    # 每种 actor 的数量范围
    six_dof:                        # 6-dof actor 的随机化范围
      center: [0, 0, 0]
      bound_position: [[-1, 1], [-1, 1], [0, 2]]
      bound_rotation: [[0, 3.14159], [0, 3.14159], [0, 3.14159]]
```

9.6.3 场景配置校验

```
# SceneManager 自动进行以下校验:
# 1. Actor 数量与名称数量一致性
# 2. Joints 数量与 joints_dof 数量一致性
# 3. 随机化参数完整性
# 4. 灯光配置有效性
# 5. 所有 spawnable 路径存在性
```

9.7 数据采集管理器

9.7.1 DataCollectionManager（`src/dataCollectionManager/data_collection_manager.py`）

DataCollectionManager 是 ORCA 仿真环境的核心管理器，负责环境生命周期、控制器管理、数据采集和模型推理。

两种模式：

模式	说明	用途
`TELECONTROL`	遥操作采集模式	通过手柄控制机器人，采集训练数据
`AUGMENTATION`	数据增强回放模式	回放已采集数据，进行数据增强

主循环流程：

```
while not shutdown:
    env.reset()
    update_scene()           # 场景生成 + 任务分配
    for aug_idx in range(aug_count):
        run_episode()        # 执行一集
        if task_success: save_data()
        else: clear_data()
```

核心方法：

方法	说明
`create_env()`	通过 `gym.register` + `gym.make` 创建环境
`run_controllers()`	依次运行所有控制器，合并控制输出
`run_episode()`	核心推理循环：`run_controllers()` → `env.step()` → `env.render()`
`_capture_initial_joint_qpos()`	采集初始关节位置，用于回放恢复
`_restore_initial_joint_qpos()`	回放时恢复机器人关节状态

主循环关键代码：

```
def run_episode(self):
    """执行一集数据采集"""
    for step in range(max_steps):
        # 1. 运行所有控制器
        ctrl = self.run_controllers()
        # 2. 推进仿真
        obs, reward, terminated, truncated, info = self.env.step(ctrl)
        # 3. 渲染
        self.env.render()
        # 4. 记录数据
        self._record_frame(obs, ctrl, info)
        # 5. 检查任务完成
        if self.task.is_success():
            break
```

9.8 第三方模型推理部署完整流程

9.8.1 OpenPI 推理集成

ORCA 支持通过 WebSocket 连接 OpenPI 策略服务器进行模型推理。

状态/动作布局（18 维）：

```
索引    名称              说明
[0:3]   l_pos_b           左臂末端位置（base 系，米）
[3:7]   l_quat_b          左臂末端四元数 xyzw
[7:10]  r_pos_b           右臂末端位置
[10:14] r_quat_b          右臂末端四元数 xyzw
[14]    l_grip_inner_norm 左夹爪内侧归一化 [0,1]
[15]    l_grip_outer_norm 左夹爪外侧归一化 [0,1]
[16]    r_grip_inner_norm 右夹爪内侧归一化 [0,1]
[17]    r_grip_outer_norm 右夹爪外侧归一化 [0,1]
```

推理流程：

```
from websocket_client_policy import WebsocketClientPolicy

# 1. 创建策略客户端
policy = WebsocketClientPolicy(ws_url="ws://localhost:8765")

# 2. 构建观测
def build_observation(env):
    # 获取末端位姿
    l_ee = get_ee_pose(env, "g1_omnipicker_ee_center_site_l")
    r_ee = get_ee_pose(env, "g1_omnipicker_ee_center_site_r")

    # 获取夹爪状态
    l_grip = get_gripper_state(env, "left")
    r_grip = get_gripper_state(env, "right")

    # 获取相机图像
    images = {
        "cam_head": env.get_camera_image("cam_head"),
        "cam_wrist_l": env.get_camera_image("cam_wrist_l"),
        "cam_wrist_r": env.get_camera_image("cam_wrist_r"),
    }

    state = np.concatenate([
        l_ee["pos"], l_ee["quat"],
        r_ee["pos"], r_ee["quat"],
        l_grip["inner"], l_grip["outer"],
        r_grip["inner"], r_grip["outer"],
    ])

    return {"state": state, "images": images, "prompt": "pick the red button"}

# 3. 推理循环
obs = build_observation(env)
action_chunk = policy.infer(obs)  # 返回 action chunk，维度 >= 18
action = action_chunk[0]          # 取第一个动作

# 4. 反归一化夹爪值
action[14] = action[14] * 3 - 1  # 将 [0,1] 映射到 (-1,2)
action[15] = action[15] * 3 - 1
action[16] = action[16] * 3 - 1
action[17] = action[17] * 3 - 1
```

9.8.2 自定义模型集成

参赛队伍可以集成任意推理框架（如扩散策略、ACT、RT 系列等），只需：

1. 实现观测构建函数（从 ORCA env 读取状态和图像）

2. 实现动作映射函数（将模型输出映射到 ORCA 执行器范围）
3. 在推理循环中调用 `env.step(action)` 执行动作

```
# 通用推理集成骨架
class CustomPolicy:
    def __init__(self, model_path):
        self.model = load_model(model_path)

    def build_obs(self, env):
        """构建观测：从 env 读取状态和图像"""
        return {"state": ..., "images": ...}

    def infer(self, obs):
        """模型推理：返回动作值"""
        return self.model.predict(obs)

    def map_action(self, action):
        """将模型输出映射到 ORCA 执行器范围 [-1, 1]"""
        return np.clip(action, -1, 1)

# 推理循环
policy = CustomPolicy("model.pt")
for step in range(max_steps):
    obs = policy.build_obs(env)
    raw_action = policy.infer(obs)
    action = policy.map_action(raw_action)
    env.step(action)
    env.render()
```

9.9 奖励函数定义

在当前 ORCA 数据采集环境中，`DataCollectionEnv.step()` 始终返回 `reward = 0.0`。数据采集不使用传统的 RL 奖励函数，而是通过**任务成功/失败判定**（`task.is_success()`）来筛选数据。

对于大赛评分，系统使用**基于条件的声明式判定**（YAML 配置 → PredicateProbe），而非传统 RL 奖励函数。评分判定逻辑详见第六章"大赛评分规则与机制"。

如果参赛队伍需要在训练中使用自定义奖励函数，可以在 `DataCollectionEnv` 的子类中重写 `step()` 方法：

```
class CustomEnv(DataCollectionEnv):
    def step(self, action):
        obs, _, terminated, truncated, info = super().step(action)
        # 自定义奖励计算
        reward = self._compute_reward(obs, info)
        return obs, reward, terminated, truncated, info

    def _compute_reward(self, obs, info):
        # 示例：距离奖励
        ee_pos = get_ee_position(self)
        target_pos = get_target_position(self)
        distance = np.linalg.norm(ee_pos - target_pos)
        return -distance  # 负距离作为奖励
```

第 10 章

一步一步接入指南

以下是参赛队伍从零开始接入评分系统的完整步骤。

10.1 步骤一：安装依赖

```
# 进入本地开发端目录
cd OrcaManipulation/src/examples/competition_scoring_client

# 安装 Python 依赖
pip install fastapi uvicorn pyyaml pydantic grpcio grpcio-tools

# 验证安装
python -c "import fastapi, uvicorn, yaml, pydantic, grpc; print('OK')"
```

10.2 步骤二：启动 ORCA 仿真环境

确保 ORCA 仿真环境已启动，并且 gRPC 服务在 `localhost:50051` 可用。

验证方法：评分服务启动时会在控制台打印发现的 site/body/joint 列表，确认包含以下关键实体：

- `${robot.base_body}` → 机器人基座（如 `g1_omnipicker_body_link1`）
- `${robot.ee_site}` → 末端执行器（如 `g1_omnipicker_ee_center_site_l`）
- `Static_Location_A_site`、`Static_Location_B_site`、`Static_Location_C_site`（检查点）
- `Group_Static_ElectricalCabinet_button01_site` 等（按钮）
- `Group_Interactive_ToolBox_base_site`（工具箱）
- `Group_Interactive_TestPencil_task_testpencil_testpoint_site` 等（工具）

10.3 步骤三：启动评分服务

> **推荐方式**：使用 ScorerClient 自动启动（见 步骤四），无需手动执行本步骤。仅当需要手动调试评分服务或使用非标准配置时，才需手动启动。

方式一：ScorerClient 自动启动（推荐）

```
# 只需在 ScorerClient 初始化时传入 team_id 和 team_token
# 评分服务会自动启动，无需手动操作
scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)
```

ScorerClient 初始化时自动执行以下操作： 1. 检查本地评分服务是否已在运行（`GET /api/status`） 2. 若未运行，自动查找并启动评分服务子进程（优先二进制，回退源码） 3. 等待最多 30 秒，直到服务就绪 4. 注册退出清理函数，程序退出时自动终止子进程

方式二：手动启动（需手动调试或非标准配置时使用）

使用源码启动

```
cd OrcaManipulation/src/examples/competition_scoring_client

# 基本启动（连接 ORCA 仿真，端口 9000）
python -m scorer_service.main --task-config configs/tasks.yaml --orca-addr
localhost:50051 --port 9000

# 如果 ORCA 在其他机器上，指定地址
python -m scorer_service.main --task-config configs/tasks.yaml --orca-addr
192.168.1.100:50051 --port 9000

# 连接中央服务器以自动上报结果
python -m scorer_service.main --task-config configs/tasks.yaml \
  --orca-addr localhost:50051 --port 9000 \
  --server-url http://8.159.130.194/ \
  --team-id team_001 --team-token your_team_token

# 使用严格配置模式（正式比赛环境，配置文件校验失败则拒绝启动）
python -m scorer_service.main --task-config configs/tasks.yaml \
  --orca-addr localhost:50051 --port 9000 --strict-config
```

使用二进制文件启动

```
# 二进制文件位于 dist/scorer_service/scorer_service.exe (Windows)
# 或 dist/scorer_service/scorer_service (Linux/macOS)

./dist/scorer_service/scorer_service \
  --task-config configs/tasks.yaml \
  --orca-addr localhost:50051 \
  --port 9000 \
  --strict-config
```

命令行参数完整参考

参数	默认值	说明
`--port`	`9000`	评分服务 HTTP 端口
`--host`	`127.0.0.1`	服务绑定地址
`--orca-addr`	`localhost:50051`	ORCA gRPC 地址
`--task-config`	`configs/tasks.yaml`	任务配置文件路径
`--no-orca`	否	离线模式（不连接 ORCA，用于单元测试）
`--backend`	`auto`	ORCA 连接后端：`auto`（自动）/ `full`（OrcaGymLocalEnv全功能）/ `light`（OrcaGymScene轻量）
`--team-id`	`local_dev`	队伍 ID
`--team-token`	`local_dev_token`	队伍 Token
`--server-url`	空	中央服务器地址（可选，用于自动上报结果）
`--local-store`	`./.orca_comp_scorer`	本地存储目录
`--strict-config`	否	配置文件哈希校验失败时直接退出（正式比赛必须开启）

`--robot-id`	空	指定机器人 profile ID（如 `g1_omnipicker`、`g1_pick`）

启动成功的标志：控制台输出类似以下信息：

```
================================================
  ORCA 大赛评分 SDK 服务
================================================
  任务配置: /path/to/configs/tasks.yaml
  ORCA 地址: localhost:50051
  服务端口: 9000
  本地存储: ./.orca_comp_scorer
  中央服务器: (未配置)

[OK] 配置文件哈希校验通过
  正在连接 ORCA 仿真 (localhost:50051) [后端: auto]...
[OK] 已连接 ORCA 仿真 - 全功能 (OrcaGymLocalEnv)

  评分面板: http://127.0.0.1:9000
  API 文档:  http://127.0.0.1:9000/docs
```

验证服务：

```
# 通过 HTTP 检查服务状态
curl http://127.0.0.1:9000/api/status
```

返回示例：

```
{
    "status": "running",
    "orca_connected": true,
    "current_attempt": null,
    "current_step": null
}
```

10.4 步骤四：编写推理代码并接入评分

将 `orca_scorer_client` 包复制到你的推理代码目录，或确保它在 Python 路径中。

4.1 创建你的推理代码文件（自动模式，推荐）

```
# my_competition.py
from scorer_client import ScorerClient

# 初始化评分客户端（自动启动评分服务）
scorer = ScorerClient(
    team_id="team_003",          # 你的队伍 ID
    team_token="token_team_003", # 你的队伍 Token
)

def run_task1():
    """执行任务一：移动巡检（自动模式）"""
    info = scorer.start_attempt("task1_inspection")
    print(f"开始任务: {info['task_name']}")

    # ========================================
    # 在这里编写你的机器人导航控制代码
    # 例如:
    #   nav = WaypointNavController(waypoints=[...])
    #   for step in range(max_steps):
    #       state = nav.update(env)
    #       apply_steering_drive(env, state.throttle, state.steering)
    #       env.step()
    #       env.render()
    # ========================================

    summary = scorer.finish()
    print(f"任务一完成: {summary['score']}/{summary['max_score']}")
    for step in summary.get("steps", []):
        status = "PASS" if step["success"] else "FAIL"
        diag = step.get("diagnosis", {})
        print(f"  {step['step_id']}: {status} | {diag.get('reason', '')}")


def run_task2():
    """执行任务二：按钮操作（自动模式）"""
    info = scorer.start_attempt("task2_button_press")
    print(f"开始任务: {info['task_name']}")

    # ========================================
    # 你的按钮操作代码
    # ========================================
```

```
    summary = scorer.finish()
    print(f"任务二完成: {summary['score']}/{summary['max_score']}")


def run_task3():
    """执行任务三：工具归纳（自动模式）"""
    info = scorer.start_attempt("task3_tool_storage")
    print(f"开始任务: {info['task_name']}")

    # ========================================
    # 你的工具操作代码
    # ========================================

    summary = scorer.finish()
    print(f"任务三完成: {summary['score']}/{summary['max_score']}")


if __name__ == "__main__":
    print("=" * 50)
    print("南方电网具身智能大赛 - 评分测试")
    print("=" * 50)

    run_task1()
    print()
    run_task2()
    print()
    run_task3()
```

4.2 创建你的推理代码文件（手动模式，兼容）

```
# my_competition_manual.py
from scorer_client import ScorerClient

# 初始化评分客户端
scorer = ScorerClient(
    team_id="team_003",
    team_token="token_team_003",
)

def run_task1_manual():
    """执行任务一：移动巡检（手动模式）"""
    info = scorer.start_attempt("task1_inspection")
    print(f"开始任务: {info['task_name']}")

    for step in info["steps"]:
        step_id = step["step_id"]
        scorer.begin_step(step_id)
        print(f"  执行步骤: {step['name']} ({step['score']}分)")

        # ========================================
        # 在这里编写你的机器人控制代码
        # 例如:
        #   if step_id == "start_move":
        #       robot.move_forward(speed=0.1)
        #       time.sleep(2)
        #   elif step_id == "pass_checkpoint_a":
        #       robot.navigate_to(checkpoint_a_position)
        #   ...
        # ========================================

        result = scorer.end_step()
        status = "PASS" if result["success"] else "FAIL"
        diag = result.get("diagnosis", {})
        print(f"    结果: {status} | {diag.get('reason', '')}")

    summary = scorer.finish()
    print(f"任务一完成: {summary['score']}/{summary['max_score']}")


def run_task2_manual():
    """执行任务二：按钮操作（手动模式）"""
    info = scorer.start_attempt("task2_button_press")
```

```
    print(f"开始任务: {info['task_name']}")

    for step in info["steps"]:
        step_id = step["step_id"]
        scorer.begin_step(step_id)

        # ==========================================
        # 你的按钮操作代码
        # ==========================================

        result = scorer.end_step()
        print(f"  {step_id}: {'PASS' if result['success'] else 'FAIL'}")

    summary = scorer.finish()
    print(f"任务二完成: {summary['score']}/{summary['max_score']}")


def run_task3_manual():
    """执行任务三: 工具归纳（手动模式）"""
    info = scorer.start_attempt("task3_tool_storage")
    print(f"开始任务: {info['task_name']}")

    for step in info["steps"]:
        step_id = step["step_id"]
        scorer.begin_step(step_id)

        # ==========================================
        # 你的工具操作代码
        # ==========================================

        result = scorer.end_step()
        print(f"  {step_id}: {'PASS' if result['success'] else 'FAIL'}")

    summary = scorer.finish()
    print(f"任务三完成: {summary['score']}/{summary['max_score']}")


if __name__ == "__main__":
    print("=" * 50)
    print("南方电网具身智能大赛 - 评分测试（手动模式）")
    print("=" * 50)

    run_task1_manual()
    print()
    run_task2_manual()
    print()
    run_task3_manual()
```

4.3 运行推理代码

```
# 自动模式（推荐）
python my_competition.py

# 手动模式
python my_competition_manual.py
```

10.5 步骤五：查看评分结果

方式一：通过评分面板 Web UI

打开浏览器访问：`http://127.0.0.1:9000/`

面板会实时显示： - ORCA 连接状态 - 当前步骤进度（PASS/FAIL/进行中） - 任务总分（得分/满分） - 历史尝试记录

方式二：通过 API 查询

```
# 查看评分服务状态
status = scorer.status()
print(status)

# 查看历史尝试
history = scorer.history()
for h in history:
    print(f"Task: {h['task_id']}, Score: {h.get('score', 0)}/{h.get('max_score',
0)}")
```

方式三：通过 Swagger 文档

打开浏览器访问：`http://127.0.0.1:9000/docs`

可交互式测试所有 API 端点。

10.6 步骤六：（可选）连接中央服务器

如果需要将成绩上报到中央服务器进行排行榜展示和评委评审，需要额外启动中央服务器。

中央服务器地址：**http://8.159.130.194/**

6.1 启动中央服务器

```
cd OrcaManipulation/src/examples/competition_scoring_server

# 方式一：直接启动
python app/main.py --host 127.0.0.1 --port 8000 --admin_token "your_admin_token"

# 方式二：使用 uvicorn 启动（开发模式带热重载）
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# 方式三：通过 CLI 启动
orca-scoring serve --port 8000 --admin_token "your_admin_token"
```

6.2 通过 Web UI 访问

打开浏览器访问：`http://127.0.0.1:8000/`

- 登录页：选择角色（管理员/评委/开发者），输入对应凭证
- 主面板：查看排行榜、队伍详情、步骤成功率、热力图
- 大赛指南：`http://127.0.0.1:8000/guide.html`
- 开发者指南：`http://127.0.0.1:8000/dev_guide.html`

6.3 通过 API 上报成绩

```
import requests

# 1. 登录获取 JWT Token
login_resp = requests.post("http://127.0.0.1:8000/api/auth/login", json={
    "role": "team",
    "identifier": "team_001",
    "token": "your_team_token"
})
jwt_token = login_resp.json()["data"]["token"]
headers = {"Authorization": f"Bearer {jwt_token}"}

# 2. 创建尝试记录
attempt_resp = requests.post("http://127.0.0.1:8000/api/v1/attempts", json={
    "task_id": "task2_button_press"
}, headers=headers)
attempt_id = attempt_resp.json()["attempt_id"]

# 3. 上报步骤结果
for step in steps:
    requests.post(f"http://127.0.0.1:8000/api/v1/attempts/{attempt_id}/steps",
                  json=step, headers=headers)

# 4. 结束尝试
requests.post(f"http://127.0.0.1:8000/api/v1/attempts/{attempt_id}/finish",
              headers=headers)
```

10.7 步骤七：提交最终成绩

1. 确保所有三个任务都已完成本地测试，确认得分。
2. 通过中央服务器 API 上报最终成绩。
3. 上传相关文件（代码、录屏视频等）到中央服务器。
4. 等待评委评审和最终成绩公布。

第 11 章

常见问题排查

11.1 评分服务相关

问题	可能原因	解决方案
所有步骤均失败	ORCA 仿真中无 G1 机器人	确认 ORCA 场景加载了 G1 模型；检查 `--robot-id` 参数
距离判定始终失败	机器人未移动	检查控制代码是否正确发送指令
速度判定失败	基座速度为 0	确认移动指令已发送且机器人开始运动
工具归位失败	工具未释放或位置不对	确认夹爪已打开且工具在工具箱区域内
`end_step()` 返回 error	评分服务未启动	确认 ScorerService 已启动
评分服务连接失败	端口错误或服务未启动	检查 `http://127.0.0.1:9000/api/status`
配置文件哈希校验失败	配置被修改	检查 `configs/tasks.yaml` 和 `configs/robots.yaml` 是否被修改
二进制文件找不到	未打包或路径错误	检查 `dist/scorer_service/` 目录；使用源码模式回退

11.2 ORCA 仿真相关

问题	可能原因	解决方案
ORCA 连接失败	gRPC 服务未启动	确认 ORCA 仿真在 50051 端口可用
缺少关键 site	场景版本不匹配	检查评分服务日志中的 site 列表
蓝色按钮判定失败	场景中缺少 button03_site	这是已知限制，联系组委会确认场景版本
机器人探测失败	场景中无匹配的命名特征	手动指定 `--robot-id` 参数

11.3 中央服务器相关

问题	可能原因	解决方案
登录失败	token 不匹配	检查 team_id 和 team_token 是否正确
"未登录"	未携带 Authorization header	确保请求包含 `Authorization: Bearer <token>`
端口被占用	8000 端口被其他程序占用	更换端口或终止占用进程
数据库锁定	高并发写入	重启服务，或迁移到 PostgreSQL

11.4 调试技巧

1. **查看评分服务日志**：启动评分服务时，控制台会输出所有 ORCA 实体的发现情况，以及每步判定详情。
2. **使用评分面板 Web UI**：`http://127.0.0.1:9000/` 可实时查看评分状态。
3. **查看历史记录**：`scorer.history()` 可查看所有历史尝试的得分情况。
4. **使用 Swagger 文档**：`http://127.0.0.1:9000/docs` 可交互式测试 API。
5. **检查配置哈希**：运行评分服务时观察控制台输出的 `[OK] 配置文件哈希校验通过` 或 `[WARN]` 消息。

第 12 章

附录

12.1 ORCA 场景关键实体清单

机器人实体（通过 `${robot.xxx}` 动态替换）

类别	占位符	默认值（g1_omnipicker）	用途
基座 Body	`${robot.base_body}`	`g1_omnipicker_body_link1`	位置/速度判定
自由关节	`${robot.free_joint}`	`g1_omnipicker_free_joint`	底盘移动
末端 Site	`${robot.ee_site}`	`g1_omnipicker_ee_center_site_l`	末端位置判定
夹爪关节 A	`${robot.finger_a}`	`g1_omnipicker_idx31_gripper_l_inner_joint1`	抓取判定
夹爪关节 B	`${robot.finger_b}`	`g1_omnipicker_idx39_gripper_l_inner_joint2`	抓取判定

场景实体（任务一）

类别	ORCA 名称	用途
检查点 A	`Static_Location_A_site`	路径经过点
检查点 B	`Static_Location_B_site`	路径经过点
设备柜 C	`Static_Location_C_site`	目标到达点

场景实体（任务二）

类别	ORCA 名称	颜色
按钮 01	`Group_Static_ElectricalCabinet_button01_site`	红色

按钮 02	`Group_Static_ElectricalCabinet_button02_site`	绿色
按钮 03	`Group_Static_ElectricalCabinet_button03_site`	蓝色
按钮 04	`Group_Static_ElectricalCabinet_button04_site`	黄色

场景实体（任务三）

类别	ORCA 名称	工具
工具箱	`Group_Interactive_ToolBox_base_site`	归位区域
验电笔	`Group_Interactive_TestPencil_task_testpencil_testpoint_site`	工具 1
螺丝刀	`Group_Interactive_Screwdriver_task_screwdriver_site`	工具 2
扳手	`Group_Interactive_Spanner_task_spanner_site`	工具 3
小手电筒	`Group_Interactive_Flashlight_task_flashlight_site`	工具 4
电工刀	`Group_Interactive_ElectriciansKnife01_task_electriciansknife01_site`	工具 5

12.2 配置参考

环境变量（ScorerService — 本地评分服务）

变量	说明	默认值
`ORCA_ROBOTS_CONFIG`	自定义 robots.yaml 路径	`configs/robots.yaml`

环境变量（中央服务器）

变量	说明	默认值
`ORCA_SCORING_CONFIGS_DIR`	配置目录路径	自动检测

`ORCA_SCORING_WEB_DIR`	前端静态文件目录	自动检测
`ORCA_SCORING_DB_PATH`	SQLite 数据库路径	`orca_competition.db`
`ORCA_SCORING_ADMIN_TOKEN`	管理员令牌	`SouthGrid`
`ORCA_SCORING_UPLOADS_DIR`	上传文件目录	`./uploads/`
`JWT_SECRET`	JWT 签名密钥	`orca-scoring-platform-secret-key-2026`
`DATABASE_URL`	PostgreSQL 连接串（设置后自动切换）	空（使用 SQLite）

命令行参数（ScorerService — 本地评分服务）

参数	默认值	说明
`--port`	`9000`	评分服务 HTTP 端口
`--host`	`127.0.0.1`	服务绑定地址
`--orca-addr`	`localhost:50051`	ORCA gRPC 地址
`--task-config`	`configs/tasks.yaml`	任务配置文件路径
`--no-orca`	否	离线模式（不连接 ORCA，用于单元测试）
`--backend`	`auto`	ORCA 连接后端：`auto`（自动）/ `full`（OrcaGymLocalEnv全功能）/ `light`（OrcaGymScene轻量）
`--team-id`	`local_dev`	队伍 ID
`--team-token`	`local_dev_token`	队伍 Token
`--server-url`	空	中央服务器地址（可选，用于自动上报结果）

`--local-store`	`./.orca_comp_scorer`	本地存储目录
`--strict-config`	否	配置文件哈希校验失败时直接退出
`--robot-id`	空	指定机器人 profile ID（覆盖默认配置）

命令行参数（中央服务器 — competition_scoring_server）

参数	默认值	说明
`--host`	`0.0.0.0`	服务绑定地址
`--port`	`8000`	服务端口
`--db`	`./data/scoring.db`	SQLite 数据库文件路径
`--admin_token`	`SouthGrid`	管理员令牌（**生产环境务必修改**）
`--configs_dir`	自动检测	配置目录路径
`--web_dir`	自动检测	前端静态文件目录
`--uploads_dir`	`./uploads/`	文件上传存储目录

启动方式：

```
# 进入中央服务器目录
cd OrcaManipulation/src/examples/competition_scoring_server

# 方式一：直接启动
python app/main.py --host 0.0.0.0 --port 8000 --admin_token SouthGrid

# 方式二：uvicorn 启动（开发模式热重载）
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# 方式三：使用 quick_deploy.sh 一键部署
bash quick_deploy.sh
```

12.3 角色与权限矩阵

功能	admin（管理员）	judge（评委）	team（开发者）
查看排行榜	✓	✓	✓（仅自己）

查看所有队伍详情	✓	✓	✗
查看本队数据	✓	✓	✓
查看步骤成功率画像	✓	✓	✓（仅本队）
查看热力图	✓	✓	✗
人工打分（0-100）	✓	✓	✗
管理队伍（增/删/改）	✓	✗	✗
管理评委（增/删/改）	✓	✗	✗
数据重置	✓	✗	✗
热重载评分配置	✓	✗	✗
编辑大赛指南	✓	✗	✗
编辑开发者指南	✓	✗	✗
编辑管理员文档	✓	✗	✗
品牌管理（Logo/标题）	✓	✗	✗
文件上传管理	✓	✗	✗（仅上传）
上报任务结果	✓	✓	✓
上报指标数据	✓	✓	✓
查看大赛指南	✓	✓	✓
查看开发者指南	✓	✓	✓
查看管理员文档	✓	✗	✗

> **技术支持**：如有问题，请联系大赛组委会
> **文档版本**：3.2
> **最后更新**：2026-08-05